Check for updates

International Journal of
HIGH PERFORMANCE
COMPUTING APPLICATIONS

Research Article

The International Journal of High Performance Computing Applications
2019, Vol. 33(4) 735–757
© The Author(s) 2019
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/1094342018816368
journals.sagepub.com/home/hpc
SAGE

# Acceleration of tensor-product operations for high-order finite element methods

**Kasia Świrydowicz, Noel Chalmers, Ali Karakus and Tim Warburton**

**Abstract**

This article is devoted to graphics processing unit (GPU) kernel optimization and performance analysis of three tensor-product operations arising in finite element methods. We provide a mathematical background to these operations and implementation details. Achieving close to peak performance for these operators requires extensive optimization because of the operators' properties: low arithmetic intensity, tiered structure, and the need to store intermediate results during the kernel execution. We give a guided overview of optimization strategies and we present a performance model that allows us to compare the efficacy of these optimizations against an empirically calibrated roofline.

**Keywords**

Finite element method, elliptic problem, hexahedral elements, matrix–vector product, GPU tensor operations, NVIDIA Tesla P100

## Introduction

State-of-the-art high-order finite element method (FEM)–based simulation codes now execute calculations on parallel machines with up to a million CPU cores. For example, the Nek5000 incompressible fluid flow simulator has successfully scaled up to a million CPU cores using an Message Passing Interface (MPI)-distributed computing model (Fischer et al., 2015). The current shift toward greater on-chip parallelism makes it imperative to focus on finer grain parallel implementation of high-order operators on next generation accelerators, such as graphics processing units. Efficient high-order finite element implementations rely heavily on tensor-product contractions (Deville et al., 2002). These elemental tensor-product operations are memory bound and require repeated sweeps through the data for each finite element operation. In this work, we demonstrate that despite their complexity, it is possible to achieve nearly maximum throughput on a current generation GPU.

In this article, we focus in particular on GPU implementations of three commonplace matrix–vector multiplications (further abbreviated as *matvecs*) arising in finite element codes: mass *matvec* (BK1.0), stiffness *matvec* (BK5.0), and stiffness *matvec* that involves de-aliasing integration (BK3.0). These problems have been formulated as a part of Center for Efficient Exascale Discretizations (CEED) codesign effort (CEED, 2017). BK1.0, BK5.0, and BK3.0 appear to be potentially well-suited for fine-grain parallelism but they are challenging to fully optimize. These benchmarks are characterized by low arithmetic intensity, namely that the ratio of floating point operations (FLOPS) to data transfer is low. The benchmarks also require non-unitarily strided memory access patterns since all threads that are processing a single element need the data computed by other threads that process this element. Finally, the operations consist of several concatenated tensor-product contractions and all threads simultaneously process the contractions in a prescribed order.

We detail several strategies that are collectively applied to maximize the computational throughput of these operations. In a sequence of computational experiments, we progressively optimize the operations by varying the amount of global memory accesses, shared memory usage, register variable usage, data padding, and loop unrolling. In this article, we use an NVIDIA Tesla P100 as the computational platform, as this is the target architecture for the CEED

Department of Mathematics, Virginia Tech, Blacksburg, VA, USA

**Corresponding author:**
Kasia Świrydowicz, Department of Mathematics, Virginia Tech, McBryde Hall, 225 Stanger Street, Blacksburg, VA 24061, USA.
Email: kswirydo@vt.edu

benchmarks. We note, however, that the same analysis may be applied for newer general-purpose graphics processing units (GP-GPUs), such as the NVIDIA Tesla V100, with only minor modifications.

To guide the optimization process, we construct a performance model that serves as a tool to evaluate the efficiency of the kernels. Because of the nature of the finite element operations, the performance of our kernels is limited by the bandwidth of the data streaming from both the global device memory and shared memory as well as finite register file and shared memory capacity. While a theoretical performance roofline, as presented by Lo et al. (2014), gives an upper bound on the capability of the hardware under ideal circumstances, we instead rely on an empirically calibrated roofline model that provides a realistic estimate of the best achievable performance.

### *Overview of published literature*

Early optimization studies of FEM operations date back to 2005 when Göddeke et al. (2005, 2007) solved elliptic Partial Differential Equations (PDEs) on two-dimensional (2-D) domains using a cluster equipped with GPU accelerators. In a later work, Göddeke et al. (2009) focused mostly on accelerating a Navier–Stokes solver for a lid-driven cavity and laminar flow over a cylinder. The study presented by Fu et al. (2014) targeted the entire FEM pipeline using an elliptic Helmholtz problem to show how the FEM codes are ported to the GPU. In addition, Fu et al. (2014) discussed strategies for accelerating conjugate gradient and algebraic multigrid solvers.

Other authors usually choose to work on improving the performance of one part of the FEM pipeline. For example, Cecka et al. (2011) and Markall et al. (2013) focused on the global assembly phase of FEMs and showed how to optimize this phase for GPU execution. Markall and coauthors emphasize that the choice of the most efficient algorithm strongly depends on the available hardware and selected programming model. In their work, Markall and coauthors considered the CPU and the GPU with OpenCL and CUDA parallel implementations. Moreover, Markall et al. (2010, 2013) argue that making the code portable between different threading systems (such as many-core and multi-core architectures) requires a high-level programming language.

In all of the above work, the greatest concern was efficiently pipelining GPU execution of FEMs with low-order discretizations. In contrast, Remacle et al. (2016) present algorithms for solving elliptic equations using high-order continuous hexahedral finite elements on GPUs. The issues associated with efficiently handling the greater complexity of high-order FEM operations highlighted in that work are amplified in the current article and refined with the use of a new roofline model.

The evaluation of the *matvec* typically accounts for the highest cost of the elliptic FEM solver (Remacle et al., 2016; Tables 2 and 3). Optimization of *matvec* performance on the GPUs appears in the literature mostly in the context of general applications. Yet a few papers focused directly on the low-order FEM *matvec*. For example, Dziekoński et al. (2017) used a conjugate gradient solver and optimized *matvec* as its important part. Dehnavi et al. (2010) and Grigoras et al. (2016) presented similar findings and provided optimization strategies. However, in our case the action of the operator is local to the element. We never assemble the global matrix and we perform the element-wise *matvec*. In the literature, this approach is known as the *matrix-free* approach. As the action of the element-wise *matvec* is a bandwidth-bound operation, assembling the matrix explicitly would require more memory usage inside the kernel and offer little cache reusage opportunities. Thus, in this work in order to optimize performance we focus solely on the matrix-free approach.

The *matvecs* used in this article can be expressed as a concatenation of tensor contractions. Related work on efficient implementation of tensor contractions for the GPUs consists of several papers. Nelson et al. (2015) applied a high-level language to formulate tensor contractions and used GPU code auto-tuning to enhance the performance. The findings were tested using several benchmark problems, including a problem derived from Nek5000 (see Fischer et al., 2008). The optimized code reached 120 GFLOPS/s on a Maxwell class GPU. Abdelfattah et al. (2016) published similar work where, in addition to specifying a high-level language to formulate tensor contractions, the authors introduced a performance model based on both the number of floating point operations and the number of bytes of global memory that need to be read and written. The performance results were compared to CUBLAS and to a parallel CPU code using 16 cores. The most efficient version of the code reached 180 GFLOPS/s on an NVIDIA Tesla K40 GPU. While those two papers focused on small tensor contractions, Liu et al. (2017) optimized large tensor contractions and used auto-tuning for performance optimization. Their paper also presented implementation details and reported global memory usage. Shi et al. (2016) also demonstrated BLAS-like optimizations of large tensor product contractions.

The performance model developed in this article is rooted in earlier work by several authors, and is most similar to the *roofline model* in Lo et al. (2014). Other performance models, such as those considered by Stratton et al. (2012) and Zhang and Owens (2011), modeled the GPU performance using benchmarks. These efforts focused on a semiautomatic way of identifying performance bottlenecks, are specifically designed to obtain a more complete understanding of the underlying hardware properties of the GPU. Such studies have appeared frequently in the literature due to lack of available complete GPU hardware documentation even though it is well-known that hardware organization strongly impacts performance (Hong and Kim, 2009; Lee et al., 2010; Wong et al., 2010). Volkov and Demmel (2008) and Volkov (2010) employed a different approach; they optimized matrix–matrix multiplication on the GPU and explained the code performance while

improving the implementation. In his 2010 paper, Volkov pointed out that, contrary to a common belief, high GPU occupancy is not necessary to obtain performance close to the peak declared by the GPU manufacturer. This observation is important for the current article. Our most efficient kernels are characterized by high usage of registers and shared memory. Thus, achieved occupancy can often be as low as 10% even though the performance of these kernels is close to the empirical roofline. Volkov's observation that the aggregate number of shared memory accesses per thread should be reduced to avoid bottlenecks motivates a final optimization for the high-order FEM operations considered in this work. Specifically, we exploit the intrinsic structure of the interpolation matrix used in the construction of the action of the mass and stiffness matrices to improve throughput by reducing the aggregate number of shared memory accesses.

While there exist multiple GPU performance models (see Konstantinidis and Cotronis, 2015 and Abdelfattah et al., 2016) which are based on various properties of the GPU hardware and the code characteristics, in this article we develop a simple and practical model. The model provides an upper bound on achievable performance for the bandwidthbound FEM operations. Our model is independent of implementation and, to our knowledge, there is no model of this type (i.e. based on the achievable throughput of device to device data copy) in the literature.

In this article, we show a full optimization process for FEM tensor product codes on NVIDIA Tesla P100. We start from a simple implementation and we improve it through a sequence of optimization steps. While many of the optimization strategies for improving the performance of GPU kernels used in this article have been discussed in multiple places (Kim et al., 2012; Li, 2016; Volkov, 2010; Volkov and Demmel, 2008), in this work we apply these optimizations in a systematic fashion in order to design and create fully optimized GPU kernels for a complex practical problem. In particular, we reduce the number of shared memory fetches and thus improve the performance of our code by using the row symmetry of the interpolation matrix **I** in BK1.0 and BK3.0 optimization. This property has been exploited before (see Don and Solomonoff, 1995), however, not in the context of the GPU code optimizations. Later in this article, we show how using this property results in a substantial performance boost for BK1.0 and BK3.0.

The remainder of this article is organized as follows. We first present the mathematical description of the three benchmark problems mentioned earlier. We then introduce the performance model we use to evaluate our kernels. We continue by explaining the implementation and optimization choices for each of the benchmark problems. Finally, we present concluding remarks and future goals. In the Appendix we explain how to obtain the source code and reproduce the results.

## Problem description

The three benchmark problems we discuss below were formulated by the CEED (see CEED, 2017). The problems are listed as *benchmark kernels*. These problems are motivated by numerical approximation of the screened Poisson equation

$$-\nabla^2 u + \lambda u = f \tag{1}$$

using a high-order FEM on hexahedral elements. In the equation, $f$ is a given forcing function and $\lambda$ is a constant. To begin we consider an unstructured mesh of a domain $\mathcal{D} \subset \mathbb{R}^3$ into $K$ hexahedral elements $D^e$, where $e = 1, \ldots, K$, such that

$$\mathcal{D} = \bigcup_{e=1}^{K} D^e \tag{2}$$

We assume that each hexahedral element $D^e$ with vertices $\{x_n, y_n, z_n\}_{n=1}^{n=8}$ is an image of the reference bi-unit cube $\hat{D}$ under a tri-linear map. We take the reference cube to be the bi-unit cube, that is

$$\hat{D} = \{(r, s, t) : -1 \leq r, s, t \leq 1\}$$

On each element we consider the variational form of the screened Poisson problem (1) on element $D^e$ by requiring that $u$ satisfies

$$\int_{D^e} \nabla v \cdot \nabla u \, dV + \lambda \int_{D^e} vu \, dV = \int_{D^e} vf \, dV \tag{3}$$

for all test functions $v \in H^1(D^e)$. We map the integrals to the reference cube $\hat{D}$ in order to write the variational form as

$$\int_{\hat{D}} \nabla v^T G^e \nabla u^e \, dV + \lambda \int_{\hat{D}} vu^e |J^e| \, dV = \int_{\hat{D}} vf |J^e| \, dV$$

where $|J^e|$ is the determinant of the Jacobian $J^e$ of the mapping from element $D^e$ to the reference cube $\hat{D}$ and $G^e = |J^e|(J^e)^T J^e$ is the scaled elemental metric tensor. The $\nabla$ operators in these integrals are now understood to be in $(r, s, t)$-space.

On the reference cube $\hat{D}$, we construct $u^e$, a high-order finite element approximation of the function $u$. Note that $u^e$ is a degree $N$ polynomial in each dimension. We use $\mathbb{Q}^N(\hat{D})$ to describe a space of all such polynomials. As a polynomial basis of $\mathbb{Q}^N(\hat{D})$, we choose the tensor product of 1-D Lagrange interpolation polynomials $\{l_i\}_{i=0}^{i=N}$ based on $N+1$ Gauss–Lobatto–Legendre (GLL) nodes on the interval $[-1, 1]$. We use $\{r_i\}_{i=0}^{i=N}$ to denote the GLL nodes in $[-1, 1]$. We employ an analogous notation for the GLL nodes in the $s$ and $t$ dimensions. Using a multi-index, we define the multidimensional basis polynomials $l_{ijk}(r, s, t)$ to be the tensor product of the 1-D Lagrange interpolating polynomials, that is

$$l_{ijk}(r, s, t) = l_i(r) l_j(s) l_k(t)$$

for all $0 \leq i, j, k \leq N$. Hence, on $\hat{D}$ we can express the polynomial $u^e$ as

$$u^e(r,s,t) = \sum_{i=0}^{N}\sum_{j=0}^{N}\sum_{k=0}^{N} u^e_{ijk} l_{ijk}(r,s,t)$$

Consequently, the coefficients $u^e_{ijk}$ are the nodal values of the polynomial $u^e$ at the GLL interpolation points $(r_i, s_j, t_k)$, that is, $u^e_{ijk} = u^e(r_i, s_j, t_k)$.

We enforce the variational form (3) for all test functions $v$ in the finite element space $\mathbb{Q}^N(\hat{D})$ by taking $v$ to be each of the basis Lagrange interpolating polynomials, that is, $v = l_{i'j'k'}(r,s,t)$ for all $0 \le i', j', k' \le N$. We then denote by $\mathbf{u}^e$ the vector of the polynomial coefficients of $u^e$, that is $\mathbf{u}^e = [u^e_{000}, u^e_{001}, \ldots, u^e_{NNN}]^T$. Evaluating (3) for each test function $v$ we obtain that $\mathbf{u}^e$ satisfies the following linear system

$$\mathbf{S}^e\mathbf{u}^e + \lambda\mathbf{M}^e\mathbf{u}^e = \mathbf{f}^e$$

where the local stiffness matrix $\mathbf{S}^e$, the local mass matrix $\mathbf{M}^e$, and the local load vector $\mathbf{f}^e$ are defined as

$$\mathbf{S}^e_{ijk,i'j'k'} = \int_{\hat{D}} (\nabla l_{i'j'k'})^T G^e \nabla l_{ijk}\, dV \tag{4}$$

$$\mathbf{M}^e_{ijk,i'j'k'} = \int_{\hat{D}} l_{i'j'k'} l_{ijk} |J^e|\, dV \tag{5}$$

$$\mathbf{f}^e_{i'j'k'} = \int_{\hat{D}} l_{i'j'k'} f |J^e|\, dV \tag{6}$$

We concatenate the local stiffness and mass matrix operators as well as local load vectors to form global unassembled versions which we denote, $\mathbf{S}$, $\mathbf{M}$, and $\mathbf{f}$, respectively. Note that since the global approximation of $u$ is continuous, the nodal values of $u^e$ along edges and vertices shared by neighboring elements will appear multiple local stiffness and mass matrix operations. A fully assembled system for the global degrees of freedom can be assembled by gathering the local contributions at each nodal point. In this article, we neglect the action of the global assembly and focus on the optimization of the local operators. The concatenated system of the local operators form the following block diagonal system, which operates on the concatenated vector of solution coefficients $\mathbf{u}$

$$\mathbf{S}\mathbf{u} + \lambda\mathbf{M}\mathbf{u} = \mathbf{f}$$

As we neglect the global assembly, these global stiffness and mass matrix operators can be applied in a matrix-free (element-wise) way and no communication is required between elements.

The benchmarks presented below consider the efficient action of either just the mass matrix $\mathbf{M}$ or the complete screened Poisson operator $\mathbf{S} + \lambda\mathbf{M}$ on each element. Benchmark Problems 1.0 and 3.0 consider the case where the integrals (4)-(5) are evaluated using a full Gauss–Legendre (GL) quadrature and Benchmark Problem 5.0 considers the case where the integrals are approximated using simply a quadrature at the GLL interpolation points.

## *Benchmark Problem 1.0 (BK1.0): Mass matrix multiplication*

The first benchmark we present involves a matrix–vector product of the high-order finite element mass matrix $\mathbf{M}$ and a corresponding vector. This operation is a component of finite element elliptic operators and some preconditioning strategies. Thus, it serves as a useful initial performance benchmark. The operation requires relatively little data transfers compared with more demanding differential operators which necessitate loading the elemental metric tensor for each node of the element, as discussed in later benchmarks.

Beginning from the integral definition of the entries of the local mass matrix $\mathbf{M}^e$ in (5) we use the fact that $\hat{D}$ is the reference cube to evaluate the integral in each dimension separately via an $N_q^{GL}$-node GL quadrature. We denote the quadrature weights and nodes in the $r$ dimension as $\{\tilde{w}_a\}_{a=1}^{a=N_q^{GL}}$ and $\{\tilde{r}_a\}_{a=1}^{a=N_q^{GL}}$, respectively, and use an analogous notation for quadrature nodes in the $s$ and $t$ dimensions. Thus we obtain

$$\begin{aligned}\mathbf{M}^e_{ijk,i'j'k'} = &\sum_{a=1}^{N_q^{GL}}\sum_{b=1}^{N_q^{GL}}\sum_{c=1}^{N_q^{GL}} \tilde{w}_a\tilde{w}_b\tilde{w}_c |J^e(\tilde{r}_a,\tilde{s}_b,\tilde{t}_c)| \\ &\cdot l_{i'j'k'}(\tilde{r}_a,\tilde{s}_b,\tilde{t}_c) l_{ijk}(\tilde{r}_a,\tilde{s}_b,\tilde{t}_c)\end{aligned} \tag{7}$$

We rewrite this expression in a matrix form by defining the interpolation operator

$$\mathbf{I}_{abc,ijk} = l_{ijk}(\tilde{r}_a,\tilde{s}_b,\tilde{t}_c)$$

for all $i,j,k = 0, \ldots, N$ and $a,b,c = 1, \ldots, N_q^{GL}$, the diagonal matrix $\mathbf{J}^e$ of weights, and the geometric data

$$\mathbf{J}^e_{abc,abc} = \tilde{w}_a\tilde{w}_b\tilde{w}_c |J^e(\tilde{r}_a,\tilde{s}_b,\tilde{t}_c)|$$

for all $a,b,c = 1, \ldots, N_q^{GL}$. Hence, we write the local mass matrix (7) compactly as

$$\mathbf{M}^e = \mathbf{I}^{\mathrm{T}}\mathbf{J}^e\mathbf{I}$$

Note that since the basis interpolation polynomials are tensor products of 1-D polynomials, we can define the 1-D interpolation matrix $\mathbf{I}^{1\mathrm{D}}$ as

$$\mathbf{I}^{1\mathrm{D}}_{ai} = l_i(\tilde{r}_a) \tag{8}$$

for all $i = 0, \ldots, N$ and $a = 1, \ldots, N_q^{GL}$. We then express the interpolation operator $\mathbf{I}$ as a tensor product of the 1-D operators, that is

$$\mathbf{I} = \mathbf{I}^{1\mathrm{D}} \otimes \mathbf{I}^{1\mathrm{D}} \otimes \mathbf{I}^{1\mathrm{D}}$$

Thus, the interpolation operation from the GLL interpolation nodes to the GL quadrature nodes can be applied using three tensor contractions, while the projection back to the GLL nodes via the transpose interpolation operator comprises three additional tensor contractions. Since the remaining operation is simply the multiplication by the diagonal matrix $\mathbf{J}^e$, no further tensor contractions are

**Algorithm 1.** Interpolation from GLL to GL nodes.

**Data:** (1) $\mathbf{q}^e$, size $N_p$; (2) Interpolation matrix $\mathbf{I}^{1\mathrm{D}}$, size $N_q^{GL} \times N_q$
**Output:** $\tilde{\mathbf{q}}^e$, size $N_p^{GL}$
**for** $c, a \in \{1, \ldots N_q\}$, $j \in \{1, \ldots, N_q^{GL}\}$ **do**
  $\hat{\mathbf{q}}^e_{cja} = \sum_{b=1}^{N_q} \mathbf{I}^{1\mathrm{D}}_{jb} \mathbf{q}^e_{cba}$ ▷ Interpolate in $b$ direction
**end for**
**for** $c \in \{1, 2, \ldots N_q\}$, $i, j \in \{1, 2 \ldots, N_q^{GL}\}$ **do**
  $\hat{\hat{\mathbf{q}}}^e_{cja} = \sum_{a=1}^{N_q} \mathbf{I}^{1\mathrm{D}}_{ia} \tilde{\mathbf{q}}^e_{cja}$ ▷ Interpolate in $a$ direction
**end for**
**for** $k, i, j \in \{1, 2 \ldots, N_q^{GL}\}$ **do**
  $\tilde{\mathbf{q}}^e_{kji} = \mathbf{J}^e_{kji} \sum_{c=1}^{N_q} \mathbf{I}^{1\mathrm{D}}_{kc} \hat{\hat{\mathbf{q}}}^e_{cji}$ ▷ Interpolate in $c$ direction, save
**end for**

**Algorithm 2.** Projection from GL to GLL nodes.

**Data:** (1) $\tilde{\mathbf{q}}^e$, size $N_p^{GL}$; (2) Interpolation matrix $\mathbf{I}^{1\mathrm{D}}$, size $N_q^{GL} \times N_q$
**Output:** $\mathbf{q}^e$, size $N_p$
**for** $k, i \in \{1, 2 \ldots, N_q^{GL}\}$, $b \in \{1, 2 \ldots, N_q\}$ **do**
  $\hat{\hat{\mathbf{q}}}^e_{kbi} = \sum_{j=1}^{N_q^{GL}} \mathbf{I}^{1\mathrm{D}}_{jb} \tilde{\mathbf{q}}^e_{kji}$ ▷ Project in $b$ direction
**end for**
**for** $k \in \{1, 2 \ldots, N_q^{GL}\}$ $b, a \in \{1, 2 \ldots, N_q\}$ **do**
  $\hat{\mathbf{q}}^e_{kja} = \sum_{i=1}^{N_q^{GL}} \mathbf{I}^{1\mathrm{D}}_{ia} \hat{\hat{\mathbf{q}}}_{kbi}$ ▷ Project in $a$ direction
**end for**
**for** $c, b, a \in \{1, 2 \ldots, N_q\}$ **do**
  $\mathbf{q}^e_{cba} = \sum_{k=1}^{N_q^{GL}} \mathbf{I}^{1\mathrm{D}}_{kc} \hat{\mathbf{q}}_{kba}$ ▷ Project in $c$ direction, save
**end for**

**Algorithm 3.** BK1.0: mass matrix multiplication.

**Data:** (1) $\mathbf{q}$, size $N_{el} \cdot N_p$; (2) Interpolation matrix $\mathbf{I}^{1\mathrm{D}}$, size $N_q^{GL} \times N_q$; (3) Scaled Jacobians, $\mathbf{J}$, size $N_{el} \times N_p^{GL}$
**Output:** $\mathbf{M}\mathbf{q}$, size $N_{el} \cdot N_p$
**for** $e \in \{1, 2, \ldots N_{el}\}$ **do**
  $\tilde{\mathbf{q}}^e = \mathrm{Interpolate}(\mathbf{q}^e, \mathbf{I}^{1\mathrm{D}})$ ▷ Interpolate to GL nodes (Algorithm 1)
  **for** $k, i, j \in \{1, 2 \ldots, N_q^{GL}\}$ **do**
    $\tilde{\mathbf{q}}^e_{kji} = \mathbf{J}^e_{kji} \tilde{\mathbf{q}}^e_{kji}$ ▷ Scale by Jacobian and integration weights
  **end for**
  $\mathbf{q}^e = \mathrm{Project}(\tilde{\mathbf{q}}^e, \mathbf{I}^{1\mathrm{D}})$ ▷ Project to GLL nodes (Algorithm 2)
**end for**

required. Since we will make use of the interpolation and projection operations again in BK3.0 below, we detail their pseudo-code in Algorithms 1 and 2. We then detail the full matrix-free action of the mass matrix in the pseudo-code in Algorithm 3.

## *Benchmark Problem 5.0 (BK5.0): Stiffness matrix with collocation differentiation*

For our second benchmark, we consider the matrix–vector product of the full high-order finite element screened Poisson operator $\mathbf{S} + \lambda\mathbf{M}$ and a corresponding vector. In this benchmark, the operators $\mathbf{S}$ and $\mathbf{M}$ are evaluated using a collocation GLL quadrature rather than the more accurate GL quadrature used in the other two benchmark problems. This operation is central to many elliptic finite element codes and is usually a part of a discrete operator we wish to invert. For example, incompressible flow solvers such as Nek5000 (see Fischer et al., 2008) require solving a Poisson potential problem at each time step. Consequently, this *matvec* is potentially evaluated many times in each time step of a flow simulation, making its optimization a significant factor for good performance.

To introduce the application of the full screened Poisson operator we begin by describing the local stiffness matrix $\mathbf{S}^e$ defined in (4). We evaluate the integral in (4) in each

dimension separately, this time using the $N+1$ GLL interpolation nodes as the quadrature. Recall that we denote the GLL quadrature weights and nodes in the $r$ dimension as $\{w_a\}_{a=0}^{a=N}$ and $\{r_a\}_{a=0}^{a=N}$, respectively, and similarly for the $s$ and $t$ dimensions. Thus we obtain

$$\mathbf{S}^e_{ijk,i'j'k'} = \sum_{a=0}^{N}\sum_{b=0}^{N}\sum_{c=0}^{N} w_a w_b w_c \Big(\nabla l_{i'j'k'}(r_a,s_b,t_c)\Big)^T \cdot G^e(r_a,s_b,t_c)\nabla l_{ijk}(r_a,s_b,t_c) \tag{9}$$

To write this expression in a more compact matrix form we begin by defining the differentiation operators

$$\begin{aligned}\mathbf{D}^r_{abc,ijk} &= \frac{\partial l_{ijk}}{\partial r}(r_a,s_b,t_c)\\ \mathbf{D}^s_{abc,ijk} &= \frac{\partial l_{ijk}}{\partial s}(r_a,s_b,t_c)\\ \mathbf{D}^t_{abc,ijk} &= \frac{\partial l_{ijk}}{\partial t}(r_a,s_b,t_c)\end{aligned} \tag{10}$$

for $i,j,k,a,b,c = 0, \ldots N$. We then define the gradient operator $\mathbf{D}$ to be the vector of these three derivative operators, that is, $\mathbf{D} = [\mathbf{D}^r, \mathbf{D}^s, \mathbf{D}^t]^T$. Next, since $G^e$ is the scaled metric tensor on element $D^e$ defined by $G^e = |J^e|(J^e)^T J^e$, we write

$$G^e = \begin{pmatrix} G^e_{rr} & G^e_{rs} & G^e_{rt}\\ G^e_{sr} & G^e_{ss} & G^e_{st}\\ G^e_{tr} & G^e_{ts} & G^e_{tt}\end{pmatrix}$$

Next, we define a matrix $\mathbf{G}^e$ of operators where each entry of $\mathbf{G}^e$ is a diagonal matrix of weights and geometric data from $G^e$. That is, we define

$$\mathbf{G}^e = \begin{pmatrix} \mathbf{G}^e_{rr} & \mathbf{G}^e_{rs} & \mathbf{G}^e_{rt}\\ \mathbf{G}^e_{sr} & \mathbf{G}^e_{ss} & \mathbf{G}^e_{st}\\ \mathbf{G}^e_{tr} & \mathbf{G}^e_{ts} & \mathbf{G}^e_{tt}\end{pmatrix} \tag{11}$$

where each entry, say $\mathbf{G}^e_{rr}$, is defined as a diagonal matrix, that is

$$(\mathbf{G}^e_{rr})_{abc,abc} = w_a w_b w_c G^e_{rr}(r_a,s_b,t_c)$$

for $a,b,c = 0, \ldots, N$. We use analogous definitions for the remaining entries of $\mathbf{G}^e$. Thus, the local stiffness matrix (9) is expressed in a compact form as follows

$$\mathbf{S}^e = \mathbf{D}^T\mathbf{G}^e\mathbf{D}$$

To simplify the action of this local stiffness matrix we again use the fact that the basis interpolation polynomials are tensor products of 1-D polynomials. We define the 1-D differentiation matrix $\mathbf{D}^{1D}$ as

$$\mathbf{D}^{1D}_{ai} = l_i(r_a)$$

for all $i,a = 0, \ldots, N$. Using this 1-D derivative operator, and the fact that the GLL quadrature nodes collocate with the interpolation nodes using the Lagrange basis polynomials $l_i$, we write the partial derivative matrices $\mathbf{D}^r, \mathbf{D}^s$, and $\mathbf{D}^t$ as tensor products of $\mathbf{D}^{1D}$ and the identity matrix $I$ as follows

$$\begin{aligned}\mathbf{D}^r &= \mathbf{D}^{1D}\otimes I\otimes I\\ \mathbf{D}^s &= I\otimes\mathbf{D}^{1D}\otimes I\\ \mathbf{D}^t &= I\otimes I\otimes\mathbf{D}^{1D}\end{aligned}$$

Thus, differentiation along each dimension can be computed using single tensor contractions.

Finally, to write the full action of the local screened Poisson operator $\mathbf{S}^e + \lambda\mathbf{M}^e$ we note that since we have used the collocation GLL quadrature in the evaluation of the integrals (4)-(5), we can follow the description of the mass matrix operator above to find that no interpolation is required and the mass matrix can be written simply as $\mathbf{M}^e = \mathbf{J}^e$ where the matrix of geometric factors $\mathbf{J}^e$ is now defined using the GLL weights and quadrature points, that is

$$\mathbf{J}^e_{abc,abc} = w_a w_b w_c |J^e(r_a,s_b,t_c)|$$

for $a,b,c = 0, \ldots, N$. The local screened Poisson operator is

$$\mathbf{S}^e + \lambda\mathbf{M}^e = \mathbf{D}^T\mathbf{G}^e\mathbf{D} + \lambda\mathbf{J}^e$$

This operator can be applied using only six tensor contractions. First, we apply the $\mathbf{D}$ operator by differentiating along each dimension using three tensor contractions. We then multiply by the necessary geometric factors $\mathbf{G}^e$ and apply the transpose operator $\mathbf{D}^T$ with three more tensor contractions. Finally we add the mass matrix contribution which requires no tensor contractions. We detail the full matrix-free action of the screened Poisson operator in the pseudo-code in Algorithm 4.

### *Benchmark Problem 3.0 (BK3.0): Stiffness matrix evaluated with quadrature*

The final benchmark we consider is the same matrix–vector product of the high-order screened Poisson operator $\mathbf{S} + \lambda\mathbf{M}$ as in BK5.0. This time, however, we use the full GL quadrature to approximate the integrals in (4) and (5). This benchmark combines computational elements from BK1.0 and BK5.0 which makes for a more arithmetically intense kernel. Thus, maximizing its performance on GPUs is challenging.

We again begin by describing the local stiffness matrix $\mathbf{S}^e$ defined in (4). We evaluate the integral in (4) in each dimension separately, this time applying the full GL interpolation nodes as the quadrature. Using the notation introduced in BK1.0 above, we write

$$\mathbf{S}^e_{ijk,i'j'k'} = \sum_{a=1}^{N_q^{GL}}\sum_{b=1}^{N_q^{GL}}\sum_{c=1}^{N_q^{GL}} \tilde{w}_a\tilde{w}_b\tilde{w}_c \Big(\nabla l_{i'j'k'}(\tilde{r}_a,\tilde{s}_b,\tilde{t}_c)\Big)^T \cdot G^e(\tilde{r}_a,\tilde{s}_b,\tilde{t}_c)\nabla l_{ijk}(\tilde{r}_a,\tilde{s}_b,\tilde{t}_c) \tag{12}$$

Note here that the quadrature requires the gradients of the basis polynomials $l_{ijk}$ to be evaluated at the GL quadrature nodes. Simply composing the interpolation and

**Algorithm 4.** BK5.0: Collocation differentiation for 3D Hexahedral Mesh.

**Data:** (1) Vector $\mathbf{q}$, size $N_{el} \times N_p$, (2) differentiation matrix $\mathbf{D}^{1\mathrm{D}}$, size $N_q \times N_q$, (3) geometric factors $\mathbf{G}$, size $N_{el} \times N_p \times 7$, (4) parameter $\lambda$ ;
**Output:** Vector $\mathbf{Sq}$, size $N_{el} \times N_p$ ;
▷ Loop over elements ↓
**for** $e \in \{1, 2, \ldots N_{el}\}$ **do**
  **for** $i, j, k \in \{1, 2, \ldots N_q\}$ **do**
    ▷ Load geometric factors ↓
    $\mathtt{Grr} = \mathbf{G}^e_{1;kji}, \mathtt{Grs} = \mathbf{G}^e_{2;kji}, \mathtt{Grt} = \mathbf{G}^e_{3;kji};$
    $\mathtt{Gss} = \mathbf{G}^e_{4;kji}, \mathtt{Gst} = \mathbf{G}^e_{5;kji}, \mathtt{Gtt} = \mathbf{G}^e_{6;kji};$
    ▷ Multiply by $\mathbf{D}$ ↓
    $\mathtt{qr} = \sum_{n=1}^{N_q} \mathbf{D}^{1\mathrm{D}}_{in} \mathbf{q}^e_{kjn};$
    $\mathtt{qs} = \sum_{n=1}^{N_q} \mathbf{D}^{1\mathrm{D}}_{jn} \mathbf{q}^e_{kni};$
    $\mathtt{qt} = \sum_{n=1}^{N_q} \mathbf{D}^{1\mathrm{D}}_{kn} \mathbf{q}^e_{nji};$
    ▷ Apply chain rule ↓
    $\mathtt{rqr}^e_{ijk} = \mathtt{Grr*qr + Grs*qs + Grt*qt};$
    $\mathtt{rqs}^e_{ijk} = \mathtt{Grs*qr + Gss*qs + Gst*qt};$
    $\mathtt{rqt}^e_{ijk} = \mathtt{Grt*qr + Gst*qs + Gtt*qt};$
  **end for**
  **for** $i, j, k \in \{1, 2, \ldots N_q\}$ **do**
    $\mathtt{J} = \mathbf{G}^e_{7;kji}$
    $\mathbf{Sq}^e_{ijk} = \lambda \mathtt{J} \mathbf{q}^e_{kji} + \sum_{n=1}^{N_q} \mathbf{D}^{1\mathrm{D}}_{ni} \mathtt{rqr}^e_{kjn} + \mathbf{D}^{1\mathrm{D}}_{nj} \mathtt{rqs}^e_{kin} + \mathbf{D}^{1\mathrm{D}}_{nk} \mathtt{rqr}^e_{nji};$
  **end for**
**end for**

differentiation operators $\mathbf{I}$ and $\mathbf{D}$ defined in BK1.0 and BK5.0 above requires nine tensor contractions to evaluate this quantity. Indeed, for each of the $r$, $s$, and $t$ derivatives of $l_{ijk}$, we need an operation which combines differentiation and interpolation to the GL quadrature along one dimension and only interpolation to the GL quadrature along the remaining two dimensions.

We instead reduce the number of required tensor contractions by considering the Lagrange interpolating polynomials $\tilde{l}_a(r)$ for $a = 1, \ldots, N_q^{GL}$ which interpolate the GL quadrature nodes. We define the tensor product basis polynomials as it is done for the GLL interpolating basis, that is

$$\tilde{l}_{abc}(r, s, t) = \tilde{l}_a(r)\tilde{l}_b(s)\tilde{l}_c(t)$$

for $a, b, c = 1, \ldots, N_q^{GL}$. We then define the derivative operators, as done above for the GLL interpolating Lagrange basis functions, on this set of polynomials as follows

$$\tilde{\mathbf{D}}^r_{abc,a'b'c'} = \frac{\partial \tilde{l}_{a'b'c'}}{\partial r}(\tilde{r}_a, \tilde{s}_b, \tilde{t}_c)$$

$$\tilde{\mathbf{D}}^s_{abc,a'b'c'} = \frac{\partial \tilde{l}_{a'b'c'}}{\partial s}(\tilde{r}_a, \tilde{s}_b, \tilde{t}_c)$$

$$\tilde{\mathbf{D}}^t_{abc,a'b'c'} = \frac{\partial \tilde{l}_{a'b'c'}}{\partial t}(\tilde{r}_a, \tilde{s}_b, \tilde{t}_c)$$

for $a, b, c, a', b', c' = 1, \ldots, N_q$. We can then construct the gradient operation on this set of basis function as $\tilde{\mathbf{D}} = [\tilde{\mathbf{D}}^r, \tilde{\mathbf{D}}^s, \tilde{\mathbf{D}}^t]^T$. Thus, if we view the interpolation operators defined in (8) as transforming a polynomial from the basis of Lagrange interpolating polynomials on the GLL nodes to the basis of Lagrange interpolating polynomials on the GL quadrature, then we can use the operator $\tilde{\mathbf{D}}$ to evaluate the derivatives of this polynomial on the GL node basis.

With these derivative operators, we continue as in BK5.0 by defining the matrix of weights and geometric data $\mathbf{G}^e$ in (11), and this time

$$(\mathbf{G}^e_{rr})_{abc,abc} = \tilde{w}_a \tilde{w}_b \tilde{w}_c G^e_{rr}(\tilde{r}_a, \tilde{s}_b, \tilde{t}_c)$$

for $a, b, c = 1, \ldots, N_q^{GL}$. We use analogous definitions for the remaining entries of $\mathbf{G}^e$. Using these matrix operators we can write the local stiffness matrix (9) compactly as follows

$$\mathbf{S}^e = \mathbf{I}^T \tilde{\mathbf{D}}^T \mathbf{G}^e \tilde{\mathbf{D}} \mathbf{I}$$

Note that, as with the GLL interpolation basis polynomials, we can define the 1-D differentiation operator $\tilde{\mathbf{D}}^{1\mathrm{D}}$ defined such that

$$\tilde{\mathbf{D}}^{1\mathrm{D}}_{ai} = \tilde{l}_i(\tilde{r}_a)$$

for $i, a = 1, \ldots N_q^{GL}$, so that each differentiation operation in $\tilde{\mathbf{D}}$ can be written as a tensor product of $\tilde{\mathbf{D}}^{1\mathrm{D}}$ and the identity matrix $I$. Therefore, the differentiation operators on the GL Lagrange interpolation basis polynomials along each dimension can be applied using a single tensor contraction.

Finally, to express the full action of the local screened Poisson operator $\mathbf{S}^e + \lambda \mathbf{M}^e$ we combine the discussion in BK1.0 regarding using the GL quadrature to evaluate the

**Algorithm 5.** BK3.0: Differentiation for 3D Hexahedral Elements.

**Data:** (1) Vector $\mathbf{q}$, size $N_{el} \times N_p$, (2) differentiation matrix $\tilde{\mathbf{D}}^{1\mathrm{D}}$, size $N_q^{GL} \times N_q^{GL}$, (3) Interpolation matrix $\mathbf{I}$, size $N_q^{GL} \times N_q$, (4) geometric factors $\mathbf{G}$, size $N_{el} \times N_p^{GL} \times 7$, (5) parameter $\lambda$ ;
**Output:** Vector $\mathbf{Aq}$, size $N_{el} \times N_p$ ;
**for** $e \in \{1, 2, \ldots N_{el}\}$ **do**
  $\tilde{\mathbf{q}}^e = \mathrm{Interpolate}(\mathbf{q}^e, \mathbf{I}^{1\mathrm{D}})$ ▷ Interpolate to GL nodes (Algorithm 1)
  **for** $i, j, k \in \{1, 2, \ldots N_q^{GL}\}$ **do**
    ▷ Load geometric factors ↓
    $\mathtt{Grr} = \mathbf{G}^e_{1;kji}, \mathtt{Grs} = \mathbf{G}^e_{2;kji}, \mathtt{Grt} = \mathbf{G}^e_{3;kji}$ ;
    $\mathtt{Gss} = \mathbf{G}^e_{4;kji}, \mathtt{Gst} = \mathbf{G}^e_{5;kji}, \mathtt{Gtt} = \mathbf{G}^e_{6;kji}$;
    ▷ Multiply by $\tilde{\mathbf{D}}$ ↓
    $\mathtt{qr} = \sum_{n=1}^{N_q^{GL}} \tilde{\mathbf{D}}^{1\mathrm{D}}_{in} \tilde{\mathbf{q}}^e_{kjn}$;
    $\mathtt{qs} = \sum_{n=1}^{N_q^{GL}} \tilde{\mathbf{D}}^{1\mathrm{D}}_{jn} \tilde{\mathbf{q}}^e_{kni}$;
    $\mathtt{qt} = \sum_{n=1}^{N_q^{GL}} \tilde{\mathbf{D}}^{1\mathrm{D}}_{kn} \tilde{\mathbf{q}}^e_{nji}$;
    ▷ Apply chain rule ↓
    $\mathtt{rqr}^e_{ijk} = \mathtt{Grr*qr + Grs*qs + Grt*qt}$;
    $\mathtt{rqs}^e_{ijk} = \mathtt{Grs*qr + Gss*qs + Gst*qt}$;
    $\mathtt{rqt}^e_{ijk} = \mathtt{Grt*qr + Gst*qs + Gtt*qt}$;
  **end for**
  **for** $i, j, k \in \{1, 2, \ldots N_q^{GL}\}$ **do**
    $\mathtt{J} = \mathbf{G}^e_{7;kji}$
    $\tilde{\mathbf{Aq}}^e_{ijk} = \lambda \mathtt{J} \mathbf{q}^{(e,GL)}_{kji} + \sum_{n=1}^{N_q^{GL}} \tilde{\mathbf{D}}^{1\mathrm{D}}_{ni} \mathtt{rqr}^e_{kjn} + \tilde{\mathbf{D}}^{1\mathrm{D}}_{nj} \mathtt{rqs}^e_{kin} + \tilde{\mathbf{D}}^{1\mathrm{D}}_{nk} \mathtt{rqr}^e_{nji}$;
  **end for**
  $\mathbf{Aq}^e = \mathrm{Project}(\tilde{\mathbf{Aq}}^e, \mathbf{I}^{1\mathrm{D}})$ ▷ Project to GLL nodes (Algorithm 2)
**end for**

local mass matrix $\mathbf{M}^e$ in order to write the local screened Poisson operator as

$$\mathbf{S}^e + \lambda \mathbf{M}^e = \mathbf{I}^T \tilde{\mathbf{D}}^T \mathbf{G}^e \tilde{\mathbf{D}} \mathbf{I} + \lambda \mathbf{I}^T \mathbf{J}^e \mathbf{I}$$

where $\mathbf{J}^e$ is defined as in BK1.0 above. This operator can be applied using a total of 12 tensor contractions. We first interpolate to the GL quadrature nodes by applying the interpolation operator $\mathbf{I}$ using three tensor contractions. We then differentiate along each dimension by applying the $\tilde{\mathbf{D}}$ operator using another three tensor contractions. We then multiply by the necessary geometric factors and multiply by the transpose derivative operator using three additional tensor contractions and add the mass matrix contributions. Finally, we use three tensor contractions to multiply by the transpose interpolation operator in each dimension to project the result back to the GLL interpolation nodes. We detail the full matrix-free action of the stiffness matrix evaluated with the numerical quadrature in pseudo-code in Algorithm 5.

## Empirical performance model

The performance for all three benchmark problems is limited by global memory bandwidth. In BK5.0 and BK3.0, we load seven geometric factors for every node in the FEM element in addition to loading and storing the field vector $\mathbf{q}$ itself. Moreover, as we emphasize in the introduction, for all three operations the data-to-FLOP ratio is high which limits our ability to hide memory latency behind computation.

Konstantinidis and Cotronis (2015) proposed a GPU performance model that accounts for the various data-to-FLOP ratios. However, the kernels used in the model achieve occupancy of around 97% while, for our kernels, extensive use of register file and shared memory results in achieved occupancy rarely exceeding 25%. Hence, we cannot apply this model for our kernels in a meaningful way.

A different idea was presented in Abdelfattah et al. (2016), where the authors used the size of global memory transfers as a means of comparison. The advantage of such approach is that it is independent of implementation. In this work, we use a model which is similar to Abdelfattah et al. (2016). However, we transfer comparatively more data due to the geometric factors. Our model is also based on an assumption that the bandwidth of the global device data transfer is the limiting factor. Thus, we compare the performance of our kernels to the performance of copying data from global GPU memory to global GPU memory. The size of the data transfer we compare to is equivalent to the size of the data moved from and to the global memory in the particular benchmark problems, see Table 2.

For example, for every element in the FEM mesh the BK1.0 code needs to read

$$R = N_p + N_p^{GL}$$

doubles and needs to write

$$W = N_p$$

doubles. Hence, the total global memory transfer is

$$T_{\mathrm{BK1.0}} = 2N_p + N_p^{GL}$$

doubles per element. Since the memory bus is bidirectional and a double variable consists of eight bytes of data, we compare the performance of our code for BK1.0 to transferring

$$8N_{el}\frac{2N_p + N_p^{GL}}{2}$$

bytes of data. Each data transfer is executed 10 times using a standard `cudaMemcpy` function and the performance in terms of GB/s is measured by taking an average of the 10 measurements.

Let $d_r$ denote the number of bytes read from the global GPU memory and $d_w$ denote the number of bytes written to global GPU memory by a GPU kernel. Let $B_{gl}$ denote global memory bandwidth for copying $\frac{d_r+d_w}{2}$ bytes of data. Let $F$ denote the number of floating point operations that must be executed in this kernel. In our model, the maximal GFLOPS/s are estimated using a formula

$$\mathcal{R}_{\mathrm{global}} = \frac{B_{gl} \cdot F}{d_w + d_r} \tag{13}$$

For BK1.0, BK5.0, and BK3.0, the roofline $\mathcal{R}_{\mathrm{global}}$ is evaluated as a function of polynomial degree $N$. Figure 1 shows maximum TFLOPS/s for BK1.0, BK5.0, and BK5.0, respectively.

The efficiency of data transfer depends on the size of data, with throughput maximized if sufficiently large amounts of data are being transferred. We therefore expect higher bandwidth for a mesh with more elements, and for higher-degree polynomial approximations. We note that for the GPU used in this article, the NVIDIA Tesla P100 12 GB PCI-E, the mesh containing 512 elements is too small to effectively hide data transfer latencies. To see this, we note that our approach is to parallelize the problem by assigning each element to a thread block. The NVIDIA Tesla P100 has 56 SMs and two blocks of threads can be processed simultaneously on every SM. The GPU is therefore capable of processing 112 blocks of threads concurrently, which for the mesh of 512 elements means only 5 thread blocks are executed per SM. The result of this small workload per SM is that the overhead cost of kernel launch is not offset by the execution time of the kernel itself and also the empirical bandwidths which we observe in these data transfers are noisy due to overhead costs.

Inspired by Volkov and Demmel (2008), for several kernels tested for BK1.0 and BK3.0., we devised a supplementary theoretical roofline based on shared memory bandwidth. We observe that in addition to copying the data to/from global memory, we also copy the data to/from shared memory.[1] Since the memory bandwidth of shared memory is lower than the register bandwidth, shared memory transactions can limit performance. We denote the shared memory bandwidth by $B_{\mathrm{sh}}$ and and estimate it with

$$B_{\mathrm{sh}} = \#\mathrm{SMs} \times \mathrm{SIMD\ width} \times \mathrm{word\ length} \times \mathrm{clock\ speed\ in\ Ghz}$$

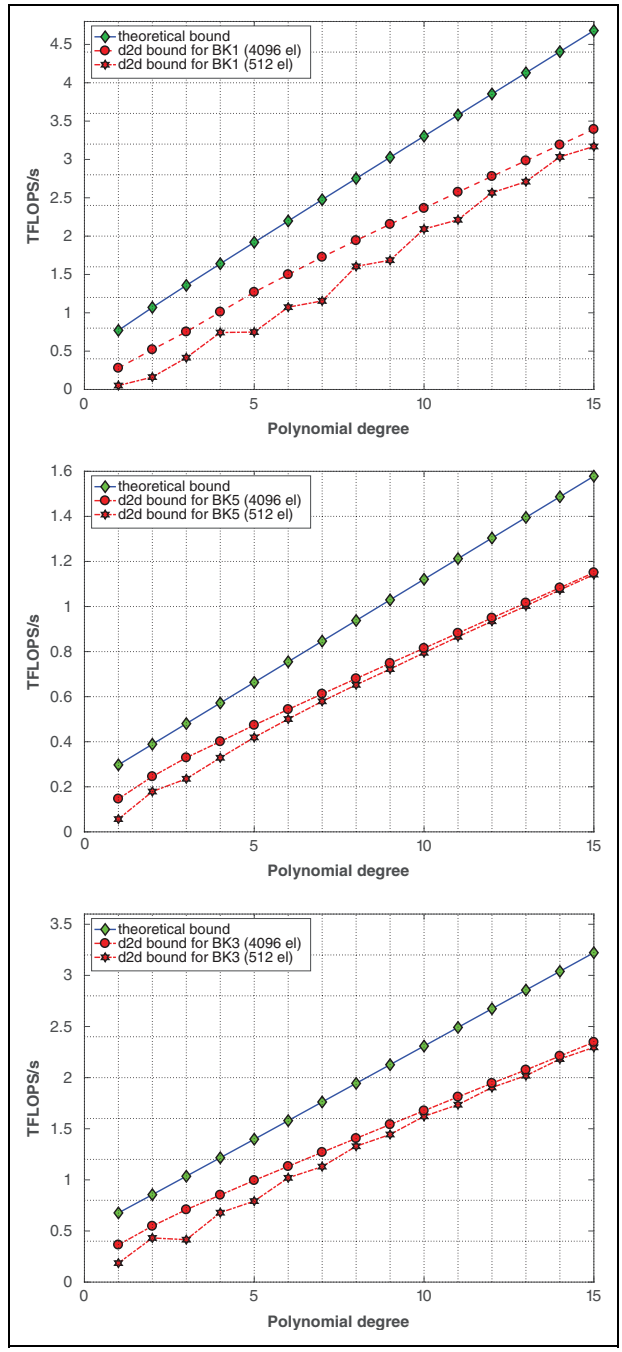


**Figure 1.** Performance roofline bounds for the BK1.0 (top), BK5.0 (middle), and BK3.0 (bottom) benchmarks. In each chart, the upper plot (line with diamond-shaped ticks) shows a theoretical bound obtained using theoretical peak bandwidth of 549 GB/s for the NVIDIA P100 PCI-E 12-GB GPU. The lower plots show the empirical peak bandwidth bound obtained using measured bandwidth attained when performing a device memory to device memory copy (upper line for a hexahedral mesh with 4096 elements and lower line for a hexahedral mesh with 512 elements). GPU: graphics processing unit.

With this ansatz we estimate that the shared memory for the NVIDIA Tesla P100 12 GB PCI-E GPU is limited to $B_{\text{sh}} = 56 \cdot 32 \cdot 4 \text{ bytes} \cdot 1.126 \text{ Ghz} = 8.071 \text{ TB/s}$. The shared memory roofline model is then given by the equation

$$\mathcal{R}_{\text{shared}} = \frac{B_{\text{sh}} \cdot F}{s_r + s_w} \tag{14}$$

where $F$ denotes the number of floating point operations performed (per thread block), $s_r$ is the number of bytes read from the shared memory (per thread block), and $s_w$ is the number of bytes written to the shared memory (per thread block).

## Notation

The notation used throughout this article is shown in Table 1. In addition, we define

$$1 \text{ TFLOPS} = 10^9 \text{FLOPS}$$

The floating point throughput rates for compute kernels in this article are reported using TFLOPS/s. We use double precision arithmetic in all the tests.

## Hardware and software

All computational studies in this article were performed on a single Tesla P100 (Pascal class) PCI-E GPU with 12 GB RAM and maximum theoretical bandwidth of 549 GB/s. The GPU is hosted by a server node equipped with Intel Xeon E5-2680 v4 processor, 2.40 Ghz with 14 cores. The code was compiled using the gcc 5.2.0 compiler and the nvcc CUDA compiler release 8.0, V8.0.61 managed by the OCCA library, version 0.2 (see Medina et al., 2014).

We compute the reference times (needed for copy bandwidth in roofline plots) using CUDA events. The kernels are timed using `MPI_Wtime`.

In our experiments, we test the code on two hexahedral cube-shaped meshes. The small mesh consists of 512 elements and the large mesh consists of 4096 elements.

## Optimizing Benchmark Problem 1.0

In this section, we describe the sequence of optimization strategies that were applied to the implementation of the BK1.0 operation.

### Kernel design

Recall that the tensor contraction operations for each element in the FEM mesh can be performed independently of other elements. Thus, to parallelize the FEM operations we assign each element to a block of threads on the GPU. In a previous work, Remacle et al. (2016) associated a single node of an element with a single thread. This is, however, impossible for high-order interpolation because if $N_q \geq 9$, we exceed the maximum number of threads per block of threads (currently limited by CUDA to 1024). Hence, for higher-order approximations we need to assign multiple nodes to a thread. We can subdivide the nodes in each element using either a 3-D thread structure or a 2-D thread structure. Figure 2 shows two such approaches. For BK1.0, we use a 2-D thread structure since we found it more effective. We also investigate a 3-D thread structure for $N_q \leq 9$ for BK5.0 and $N_q < 9$ for BK3.0.

In BK1.0, the action of the mass matrix $\mathbf{M}$ on each element can be applied using six tensor contraction operations. Hence, Algorithm 3 consists of six contractions wherein we cycle through the entire $\mathbf{q}^e$. Block-wise synchronization is needed between the contractions to ensure that the previous operation has completed. At the minimum, we need to enforce this block synchronize five times. Using a 2-D thread structure requires more thread synchronizations because we process only a slice of the nodes at a time.

### BK1.0 thread memory optimization

Because the 1-D interpolation matrix $\mathbf{I}^{1D}$ is used by all the threads in the block, we load $\mathbf{I}^{1D}$ into the shared memory.

**Table 1.** The notation used in the article.

| Symbol | | |
|---|---|---|
| Code | Math | Meaning |
| N | $N$ | Degree of the polynomial used in the interpolation |
| Nq | $N_q$ | Number of GLL quadrature nodes: $N_q = N + 1$ |
| Np | $N_p$ | Number of GLL nodes in a hexahedral element: $N_p = N_q^3$ |
| glNq | $N_q^{GL}$ | Number of GL quadrature nodes: $N_q^{GL} = N + 2$ |
| glNp | $N_p^{GL}$ | Number of GL nodes in a hexahedral element. $N_p^{GL} = (N_q^{GL})^3$ |
| I | $\mathbf{I}^{1D}$ | $(N+2) \times (N+1)$ interpolation matrix from GLL nodes to GL nodes |
| D | $\mathbf{D}^{1D}$ | $(N+1) \times (N+1)$ differentiation matrix used in BK5.0, see (10) |
| glD | $\tilde{\mathbf{D}}^{1D}$ | $(N+2) \times (N+2)$ differentiation matrix, used in BK3.0 |
| NElements | $N_{el}$ | Number of elements in the mesh |

GLL: Gauss–Lobatto–Legendre; GL: Gauss–Legendre.
[a]The first column shows the symbols used in pseudocode listings and the second column shows the symbols used in the derivations.

**Table 2.** The minimum number of doubles read and written to the global GPU memory per element in the three problems considered in this article.

| | BK1.0 | BK5.0 | BK3.0 |
|---|---|---|---|
| Read (R) | $N_p + N_p^{GL}$ | $8N_p$ | $N_p + 7N_p^{GL}$ |
| Write (W) | $N_p$ | $N_p$ | $N_p$ |
| Total bytes (T) | $2N_p + N_p^{GL}$ | $9N_p$ | $2N_p + 7N_p^{GL}$ |

GPU: graphics processing unit.

**Table 3.** Summary of the optimization strategies applied for BK1.0.

| Kernel | Description |
|---|---|
| Kernel 1 | Baseline kernel. 2-D thread structure. Intermediate result is stored in global memory. One shared array used for **I**. |
| Kernel 2 | Kernel 1 + global memory arrays used for intermediate results are replaced by two shared memory arrays. |
| Kernel 3 | Kernel 2 + the field variable **q** is cached to register arrays. |
| Kernel 4 | Kernel 3 + all the variables that are not altered by the kernel are declared using `const`. |
| Kernel 5 | Kernel 4 + padding. All shared arrays are padded if $N_q = 8.16$ and $N_q^{GL} = 8.16$. |
| Kernel 6 | Kernel 5 + loop unrolling. |
| Kernel 7 | Kernel 6 restructured; more shared memory but less thread synchronizations. |
| Kernel 8 | Kernel 7 with reduced shared memory loads using the symmetry of the interpolation operator. |

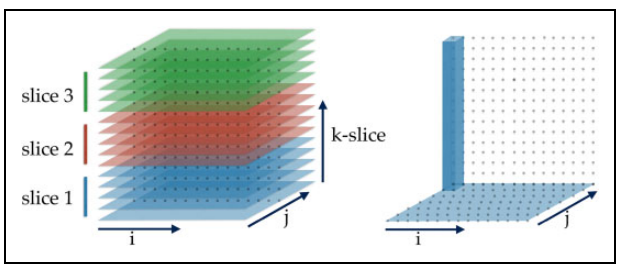


**Figure 2.** 3-D versus 2-D thread structure. On the left: 3-D approach—each thread processes a "slice" of nodes. On the right: 2-D approach–each thread processes a vertical "column" of nodes.

For the field variable **q**, we can either load $\mathbf{q}^e$ to shared memory, fetch it to registers, or fetch it piece by piece from global memory when needed. Note that we also need a placeholder array to store the partial results between the loops. There are several options to choose from, however, only a single array of size $N_p^{GL}$ can be stored in shared memory. Storing two such arrays is not feasible since we exceed the limit of 48 KB shared memory for thread block for a large $N$.

## BK1.0 kernel optimization

We show the performance results of eight GPU kernels in Figure 3. The eight kernels were constructed in sequence beginning from a direct implementation of the pseudo-code in Algorithm 3 and applying successive optimizations. The results shown in Figure 3 help to demonstrate the effect each optimization has on the overall performance of BK1.0. We list below some details of each kernel here as well as any optimization made in that kernel. Unless otherwise noted, Kernel $n$ contains all the optimizations contained in Kernel $n-1$.

*Kernel 1.* This kernel serves as a reference implementation. It uses a 2-D thread structure associated with horizontal $(i,j)$ slices (see right side of Figure 2). We declare two additional global memory variables for storing intermediate results. Kernel 1 only uses shared memory for the interpolation matrix. In all the loops, it reads from and writes to global memory. As a result, the reference kernel is highly inefficient, reaching only 80 GFLOPS/s even for the larger mesh.

*Kernel 2.* In this kernel, the global auxiliary arrays are replaced by two shared memory arrays (with $(N_q^{GL})^2$ elements in each array). While processing $\mathbf{q}^e$, we read directly from the input arrays, without caching to shared memory or registers. Due to the reduced number of global memory fetches, the performance improves by approximately a factor of two.

*Kernel 3.* In this kernel, each thread allocates a register array of size $N_q^{(GL)}$ and copies a section of $\mathbf{q}^e$ to the array at the beginning of the kernel. This kernel reaches one TFLOP/s in the best case.

*Kernel 4.* In this kernel, all the input variables, except the variable to which the output is saved, are declared as `const`. This yields only a marginal improvement in performance.

*Kernel 5.* For this kernel, if $N_q = 8$ or 16 or $N_q^{GL} = 8$ or 16, we pad the shared memory arrays used for storing $\mathbf{I}^{1D}$ and partial results to avoid bank conflicts. There is only a noticeable improvement for $N = 15$ for the smaller mesh and $N = 14$ for the larger mesh.

*Kernel 6.* In this kernel, all loops including the main loop in which we process the $(i,j)$ slices are unrolled. Unrolling the $(i,j)$ loop is important for the performance, since it gives the scheduler more freedom and more opportunity for instruction-level parallelism. At this point, the performance exceeds 1 TFLOP/s for the small mesh and 1.4 TFLOPS/s for the large mesh.

*Kernel 7.* All kernels presented thus far have required $5N_q^{GL} + 1$ thread synchronizations in a block. For example, with $N = 12$, for which $N_q^{GL} = 14$, the number of synchronizations is 71. In this kernel, the number of thread synchronizations is reduced to five by allocating more shared memory. That is, we load the entire $\mathbf{q}^e$ array to shared memory as opposed to only loading $\mathbf{q}^e$ slice by slice. Figure 4 illustrates the idea behind reducing synchronizations. Kernel 7 brings the performance up to 2.25 TFLOPS/s.

*Kernel 8.* Volkov (2010) emphasized that shared memory is much slower than using registers. Hence, reducing total number of read and write requests to/from shared memory and replacing them with register read/write instructions can significantly improve performance. Kernel 8 exploits the structure of the matrix $\mathbf{I}^{1D}$ to reduce the number of shared memory transactions. Specifically, each entry in $\mathbf{I}^{1D}$

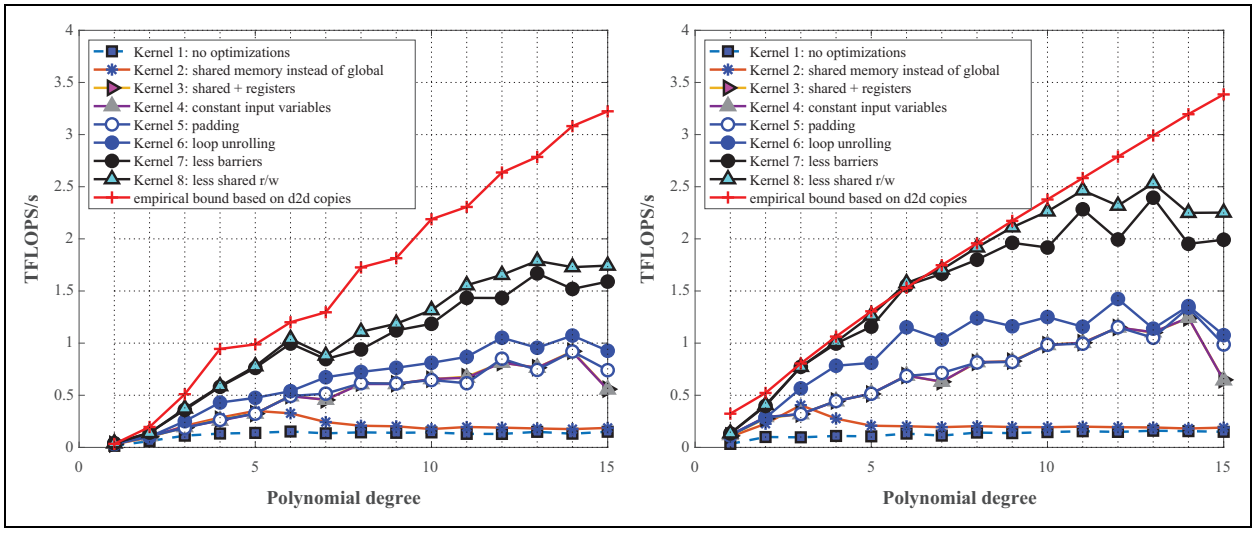


**Figure 3.** BK1.0: Achieved floating point performance. Left: results obtained using cube-shaped mesh with 512 elements. Right: results obtained using cube-shaped mesh with 4096 elements on an NVIDIA P100 PCI-E 12-GB GPU. GPU: graphics processing unit.

appears twice: $\mathbf{I}^{\mathbf{1D}}_{nm} = \mathbf{I}^{\mathbf{1D}}_{N_q^{GL}-n+1,N_q-m+1}$. Exploiting this symmetry lets us halve the number of loads from $\mathbf{I}^{\mathbf{1D}}$. This kernel loads the entire $\mathbf{I}^{\mathbf{1D}}$ into shared memory and inside each loop an appropriate entry is copied from shared memory to a register once and used twice. Since in BK1.0 we need to multiply by $\mathbf{I}^{\mathbf{1D}}$ and by $(\mathbf{I}^{\mathbf{1D}})^T$, pre-fetching only a half of matrix $\mathbf{I}^{\mathbf{1D}}$ would complicate the code and require a set of additional if-statements, which cause thread divergence. The resulting reduction in shared memory operations brings the measured performance close to the empirical roofline. The optimization strategies are summarized in Table 3.

## *BK1.0 results*

The performance numbers presented in Figure 3 reveal that the kernels considered can be categorized into three groups. Kernels 1 and 2 form the first group, Kernels 3–6 form the second group, and Kernels 7 and 8 form the third group. Between each of these three groups we observe significant performance improvements. The first major improvement occurs for Kernel 3 and is the result of caching $\mathbf{q}^e$ to register arrays prior to contraction. The second significant improvement occurs for Kernel 7 and is due to reducing the number of barriers. The performance of our reference kernel, Kernel 1, in the best case reaches only 80 GFLOPS/s, whereas our best kernel, Kernel 8, achieves 2.5 TFLOPS/s, yielding a 31-fold speedup. To obtain more than two TFLOPS/s, we needed to change the algorithm to account for the limitations intrinsic to the GPU, namely the cost of many thread synchronizations.

To explain the improvement between Kernel 6 and Kernel 8, we consider the roofline model based on shared memory bandwidth. We generate a new roofline plot by taking a minimum of the empirical roofline based on device to device copy bandwidth (13) and the upper limit computed based on shared memory bandwidth (14). We show these new rooflines in Figure 5. For Kernel 8, this roofline

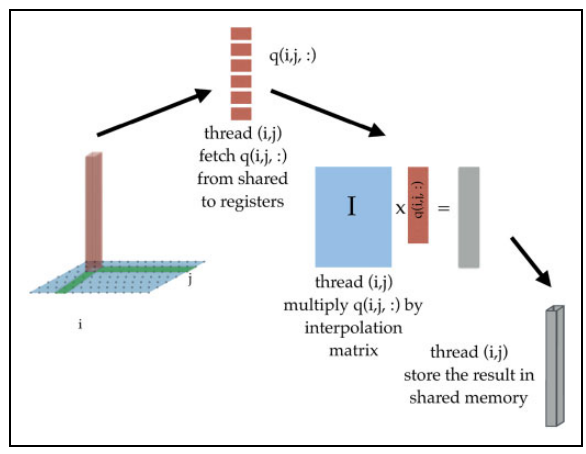


**Figure 4.** BK1.0: The idea behind reducing synchronizations in kernel 7. We fetch pieces of $\mathbf{q}^e$ to registers from shared memory and then write the result to shared memory. This does not create race conditions because we do use a 2-D thread structure and interpolate only in one direction at a time.

is identical with the global memory roofline as in Figure 3 as the shared memory roofline is higher than the global memory roofline due to eliminating almost a half of shared memory transactions. This indicated that the shared memory bandwidth is not the limiting factor of the performance of Kernel 8 for high $N$ where performance begins to degrade. For Kernels 6 and 7 however, we see that this shared memory roofline model indeed provides a reliable performance bound and these kernels are likely limited by shared memory performance. Overall, our best performing kernel is Kernel 8. The performance results for this kernel are very close to the roofline model for $N = 11$. Output from the NVIDIA profiler indicates that this kernel's performance is bound almost entirely by memory bandwidth.

Table 4 presents performance details for all the kernels with $N = 11$. We use $N = 11$ because our code achieved

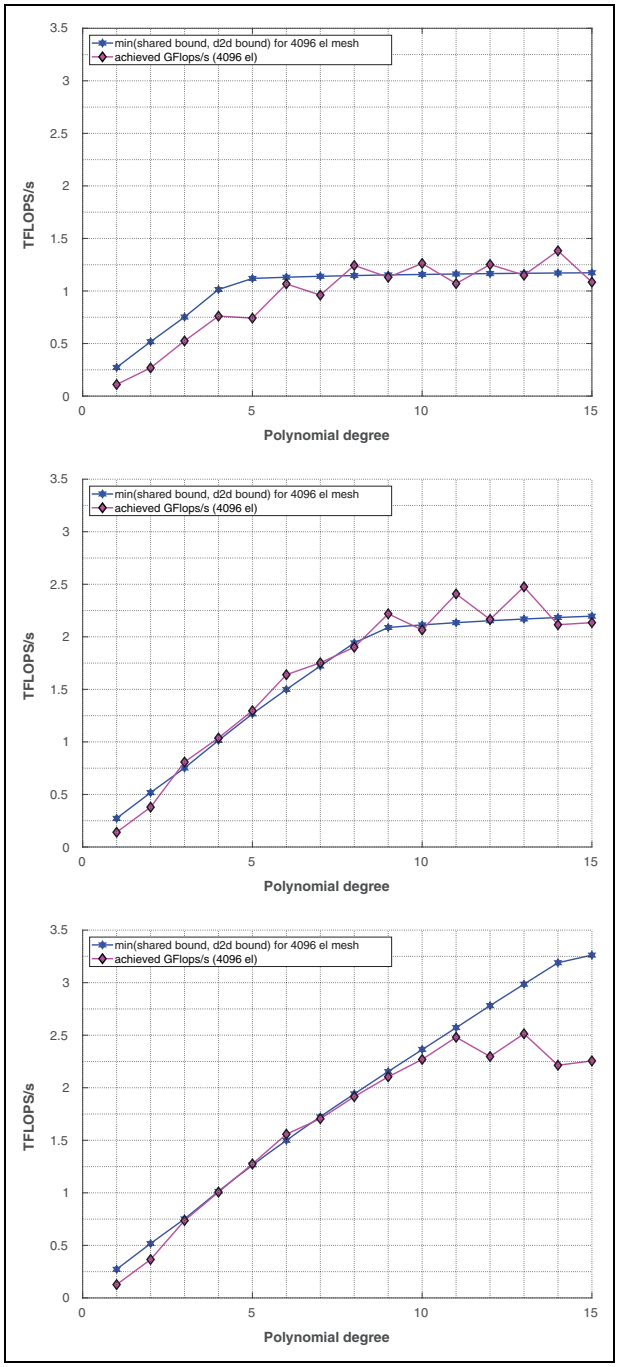


**Figure 5.** BK1.0: Shared memory rooflines and achieved floating point performance. Top: kernel 6; middle: kernel 7; bottom: kernel 8.

close to peak performance for this degree. The table shows differences in resource utilization for all the kernels.

## Optimizing Benchmark Problem 5.0

### *Kernel design*

As in BK1.0, we parallelize the action of the screened Poisson operator by assigning each element to a separate thread block on the GPU. In this optimization procedure, we investigate a 2-D thread structure as well as test a 3-D thread structure for $N < 10$.

The action of the screened Poisson operator presents a difficult optimization challenge. In particular, hiding global memory latency is significantly more important than in BK1.0 because we transfer seven geometric factors ($\mathbf{J}^e$ and six factors of $\mathbf{G}^e$) per node from global memory in every element. The majority of these values are required during the action of the stiffness matrix. To hide global memory latency, we aim to overlap data transfer with computations.

The increased amount of data that we are required to load increases both the number of global memory read transactions and the number of registers needed per thread. In theory, the geometric factors can be computed "on the fly" inside the kernel using the element's vertices and the each node's $(r_i, s_j, t_k)$ coordinates. This approach reduces the amount of double precision values loaded from global memory to $18 + 2N_q$ per block but increases the number of registers. We implemented and extensively tested this approach but concluded that it was impractical. Indeed, it is faster to simply load geometric factors from global memory than use extra registers and FLOPS.

### *BK5.0 kernel optimization: 2-D thread structure*

We show the performance results of 10 GPU kernels in Figure 7. The 10 kernels were constructed in sequence beginning from a direct implementation of the pseudocode in Algorithm 6 and applying successive optimizations. The results in Figure 7 help to demonstrate the effect each optimization has on the overall performance of BK5.0. As done with BK1.0, we list below some details of each kernel as well as any optimization made in that kernel.

*Kernel 1.* This kernel is a direct implementation of Algorithm 6. The 1-D differentiation matrix $\mathbf{D}^{1D}$ is fetched to shared memory at the beginning of the kernel. This kernel also uses a shared memory array of size $N_p$ to store partial sums. Entries of $\mathbf{q}^e$ are fetched from global memory as needed. In the last loop, the partial sums are successively added directly to the global memory array to store the final result. The kernel achieves 200 GFLOPS/s, which is one sixth of the predicted empirical roofline.

*Kernel 2.* In this kernel, all the variables, except the array used for storing the final result, are declared using `const`. This optimization has only a marginal influence on the performance for $N \geq 8$.

*Kernel 3.* In this kernel, all loops with $k$ are unrolled. Unrolling loops improves the performance for the larger mesh only if $N \geq 8$ and for all $N$ for the smaller mesh. Note that a kernel with unrolled loops uses more registers per thread and, hence, the achieved occupancy decreases. The measured TFLOPS/s increase, however, as shown in Figure 6. This behavior can be explained if unrolling has increased instruction-level parallelism.

**Table 4.** Details of performance analysis for BK1.0 and $N = 11$ on a mesh with 4096 elements.[a]

| | Registers | Shared (kB) | Shared load (GB/s) | Shared store (GB/s) | Stall reason | GFLOPS/s | % of roofline |
|---|---|---|---|---|---|---|---|
| Kernel 1 | 32 | 1.3203 | 427.84 | 639.88 | DR (45.51%) | 129.30 | 5.1 |
| Kernel 2 | 32 | 3.9609 | 1123.8 | 44.22 | OR (32.72%) | 195.78 | 7.7 |
| Kernel 3 | 64 | 3.9609 | 5934.1 | 233.47 | ED (29.96%) | 1042.94 | 40.8 |
| Kernel 4 | 64 | 3.9609 | 5979.7 | 235.26 | ED (30.85%) | 1046.68 | 41.0 |
| Kernel 5 | 58 | 3.8594 | 6372.9 | 217.27 | ED (42.71%) | 970.82 | 38.0 |
| Kernel 6 | 84 | 3.8594 | 7068.5 | 240.99 | ED (52.05%) | 1068.31 | 41.8 |
| Kernel 7 | 72 | 18.383 | 5632.8 | 972.96 | ED (39.19%) | 2247.58 | 88.0 |
| Kernel 8 | 88 | 18.383 | 3850.1 | 1169.98 | DR (22.62%) | 2478.23 | 97.0 |

DR: waiting for data request; OR: other reasons; ED: execution dependency.
[a]The *stall reason* column shows the most common reason for stalls. The best performing kernel, kernel 8, optimizes shared memory read transactions. The achieved shared memory read bandwidth drops, but the number of transactions is halved. As the optimization progresses, the kernels become less data bound.

**Algorithm 6.** BK5.0: Starting point of the implementation (2D Thread Structure).

1: **Data:** (1) Vector $\mathbf{q}$, size $N_{el} \times N_p$, (2) differentiation matrix $\tilde{\mathbf{D}}^{1\mathrm{D}}$, size $N_q \times N_q$, (3) geometric factors $\mathbf{G}$, size $N_{el} \times N_p^{GL} \times 7$, (4) parameter $\lambda$ ;
2: **Output:** Vector $\mathbf{Sq}$, size $N_{el} \times N_p$ ;
3: **for** $e \in \{1, 2, \ldots, N_{el}\}$ **do**
4: **for** each thread i,j **do**
5: Load $\mathbf{D}^{1\mathrm{D}}$ to shared memory variable `s_D`. ▷ Synchronize threads.
6: Allocate one shared auxiliary array `s_tmps[Nq][Nq][Nq]` and three register arrays: `r_Aq[Nq]`, `r_qt[Nq]` and `r_tmpt[Nq]` for storing intermediate results;
7: $\texttt{r_qt}_k = \sum_{n=1}^{N_q} \texttt{s_D}_{kn} * \mathbf{q}^e_{kji}$;
8: **for** $k \in \{1, 2, \ldots, N_q\}$ **do**
9: Load geometric factors to local variables `G00`, `G01`, `G02`, `G11`, `G12`, `G12`, `G22`, `GwJ`
10: Declare variables `qr` and `qs` and set them to `0`.
11: $\texttt{qr} = \sum_{n=1}^{N_q} \texttt{s_D}_{in} * \mathbf{q}^e_{kjn}$;
12: $\texttt{qs} = \sum_{n=1}^{N_q} \texttt{s_D}_{jn} * \mathbf{q}^e_{kni}$;
13: $\texttt{Sqtemp}_{kji} = \texttt{G00*qr} + \texttt{G01*qs} + \texttt{G02*r_qt}_k$;
14: $\texttt{s_tmps}_{kji} = \texttt{G01*qr} + \texttt{G11*qs} + \texttt{G12*r_qt}_k$;
15: $\texttt{r_tmpt}_k = \texttt{G02*qr} + \texttt{G12*qs} + \texttt{G22*r_qt}_k$;
16: $\mathbf{Sq}^e_{kji} = \mathbf{Sq}^e_{kji} + \texttt{GwJ}*\lambda*\mathbf{q}^e_{kji}$;
17: $\mathbf{Sq}^e_{kji} = \sum_{n=1}^{N_q}; \mathbf{Sq}^e_{nji} * \texttt{r_tmpt}_k$;
18: **end for** ▷ Synchronize threads.
19: **for** $k \in \{1, 2, \ldots, N_q\}$ **do**
20: Declare variables `Sq1` and `Sq2` and set them to `0`.
21: $\texttt{Sq1} = \sum_{n=1}^{N_q} \texttt{Sqtemp}_{kjn} * \texttt{s_D}_{ni}$;
22: $\texttt{Sq2} = \sum_{n=1}^{N_q} \texttt{Sqtemp}_{kni} * \texttt{s_D}_{nj}$;
23: $\mathbf{Sq}^e_{kni} = \mathbf{Sq}^e_{kni} + \texttt{Sq1} + \texttt{Sq2}$;
24: **end for** ▷ Synchronize threads.
25: **end for**
26: **end for**

*Kernel 4.* In this kernel, we place the $k$ loop (lines 7–17 in Algorithm 6) on the exterior of $i$ and $j$ loops. Doing this, $k$ becomes the slowest running index. This loop structure is justified because we iterate through $(i,j)$ slices.

*Kernel 5.* In this kernel, the auxiliary shared memory array of size $N_p$ is replaced by two auxiliary shared memory array of size $N_q^2$ each.

*Kernel 6.* In this kernel, the field variable $\mathbf{q}^e$ is fetched to the shared memory. The overall improvement is modest and the performance reaches 0.5 TFLOP/s at this point.

*Kernel 7.* This kernel reduces total global memory transactions by caching the necessary data at the beginning of the kernel and only writing the output variable once. This produces a significant optimization and improves performance by about 40%.

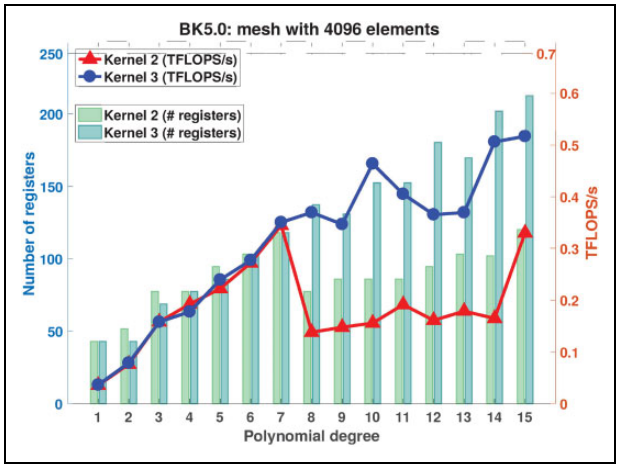


**Figure 6.** BK5.0: Comparison of performance for kernels 1 and 2. The only difference between these two kernels is loop unrolling. The number of registers per thread is shown in the bar chart in the background.

*Kernel 8.* For this kernel, if $N = 7$ or 15, the arrays are padded to avoid shared memory bank conflicts. The improvements are significant for the larger mesh and $N = 15$.

*Kernel 9.* In this kernel, each thread allocates a register array with $N_q$ elements and the field variable $\mathbf{q}^e$ is fetched to these registers instead of shared memory. For both meshes, this approach slightly improves the performance for $N = 12$ and $N = 15$.

*Kernel 10.* This kernel uses three 2-D shared memory arrays for partial results and fetches the field variable $\mathbf{q}^e$ to register arrays. The achieved TFLOPS/s for this kernel are aligned with the empirical roofline. The optimization strategies are summarized in Table 5.

## BK5.0 results: 2-D thread structure

Many of the performance improvements we observe when optimizing the kernels for BK5.0 are due to subtle changes in the code, for example, declaring variables as constant, adding padding, or unrolling the loops. But global and shared memory usage has the biggest impact on the performance. Once we reduce the use of global memory (starting in Kernel 7), the performance improves substantially, especially for larger $N$. Reducing the amount of shared memory and caching the data to registers result in the second most important factor which is visible in Figure 7 for the mesh with 4096 elements and $N \geq 10$.

Overall, we find the best performing kernel is Kernel 10. The performance of this kernel aligns well with the roofline model for $N = 10$, $N = 11$, and $N = 15$. For the reference kernel, Kernel 1, the performance barely reaches 200 GFLOPS/s, whereas the most optimized kernel we present, Kernel 10, achieved up to 1.2 TFLOPS/s. While this is only a six-fold speedup, data output from the NVIDIA profiler and comparison with our empirical roofline model based on streaming global device memory indicate that the performance of Kernel 10 is comparable to just streaming the minimally necessary data.

Table 6 presents performance details for all the kernels with $N = 11$. We use $N = 11$ because our code achieved close to peak performance for this degree. The table shows differences in resource utilization for all the kernels.

## BK5.0 kernel optimization: 3-D thread structure

We show the performance results of six GPU kernels in Figure 7. We again construct these kernels as a sequence of successive optimizations beginning from the pseudo-code shown in Algorithm 7 which uses a 3-D thread structure. We present results for these kernels for $N = 1, 2, \ldots, 9$, since associating one thread with one node as done in this

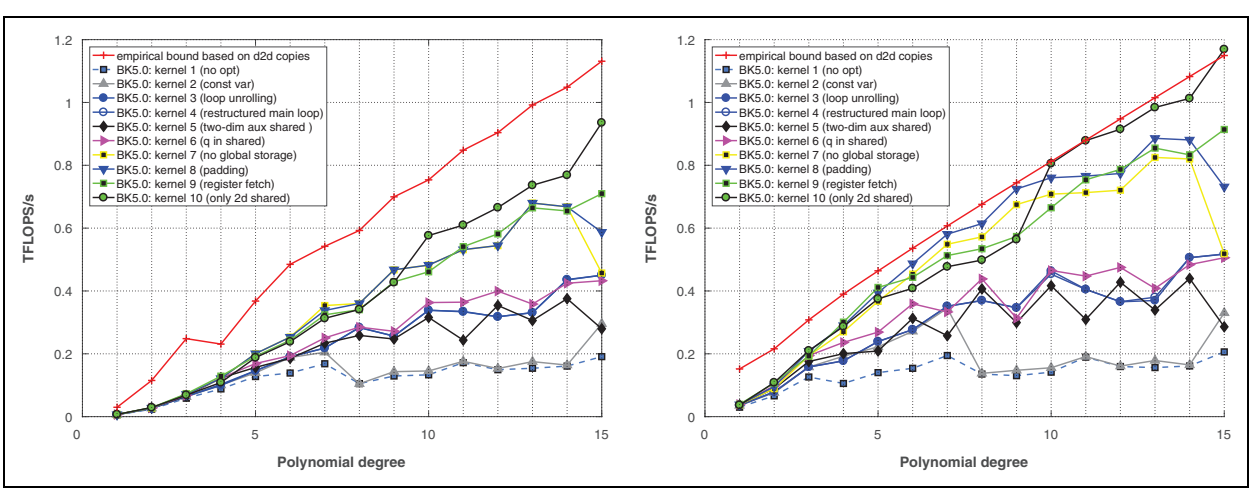


**Figure 7.** BK5.0: Performance of 2-D thread array kernels in various stages of optimization. The red line marked with crosses is the empirically determined roofline based on achievable device-to-device memory copies on an NVIDIA P100 PCI-E 12-GB GPU. Left: TFLOPS/s for cubical mesh with 512 elements. Right: TFLOPS/s for cubical mesh with 4096 elements. GPU: graphics processing unit.

**Table 5.** Summary of code optimization strategies applied for BK5.0 (2-D thread structure).

| Kernel # | Type of optimization |
|---|---|
| Kernel 1 | Baseline kernel. The matrix **D** is fetched to shared memory. We declare one shared memory array for partial result. |
| Kernel 2 | Kernel 1 + variables that are not being changed in the kernel are declared as `const`. |
| Kernel 3 | Kernel 2 + loop unrolling. |
| Kernel 4 | Kernel 3 with restructured main loop ($k$ is the slowest running index). |
| Kernel 5 | Kernel 4 with restructured shared memory. A single shared memory array of size $N_q^3$ is replaced by two arrays of size $N_q^2$ each. |
| Kernel 6 | Kernel 5 with **q** loaded to shared memory. |
| Kernel 7 | Kernel 6 + reduced global memory usage. The necessary data are read from global memory only once. The output is written once as well. |
| Kernel 8 | Kernel 7 + padding. All shared memory arrays of size $N_q^2$ are padded if $N_q = 8$ or $N_q = 16$. |
| Kernel 9 | Kernel 8 with **q** cached to registers, not to shared memory. |
| Kernel 10 | Restructured shared memory. The kernel uses three shared memory arrays, size $N_q^2$ each. |

3-D thread structure would require more than the maximum of 1024 threads for higher-degree polynomials. We list below some details of each kernel as well as any optimization made in that kernel.

*Kernel 1.* This kernel serves as an initial reference kernel and is a direct implementation of Algorithm 7. The differentiation matrix $\mathbf{D}^{1D}$ is loaded into shared memory and partial sums are stored in global arrays. The performance of this kernel reaches 300 GFLOPS/s in the best case.

*Kernel 2.* In this kernel, all variables, except the output variable, are declared using `const`. All loops with index $n$ are unrolled. The overall improvement is modest.

*Kernel 3.* In this kernel, the geometric factors are stored in registers and fetched only once from global memory. This reduces the number of global memory loads from nine to seven. This kernel reaches about 350 GFLOPS/s for the larger mesh. In case of the smaller mesh, performance of this kernel is similar to Kernel 2.

*Kernel 4.* Instead of writing the result directly to `Aq`, in this kernel the partial results are accumulated in a register variable. The performance of this kernel is only slightly better than that of Kernel 3.

*Kernel 5.* In this kernel, the field variable $\mathbf{q}^e$ is cached to shared memory. The same shared array is used to store the partial result, hence there is no need to use `Sqtemp`. The performance increases significantly and reaches 610 GFLOPS/s.

*Kernel 6.* In this kernel, partial results are stored in three shared memory arrays of size $N_p$ each. This strategy reduces occupancy but allows the kernel to use less synchronizations (the number of synchronizations reduced to two). However, there is almost no performance difference between this and the previous kernel due to extensive use of shared memory. The optimization strategies are summarized in Table 7.

## BK5.0 results: 3-D thread structure

The optimization results are shown in Figure 8. The kernels adapting 3-D thread structure form two groups. First group consists of Kernels 1–4 and the second one consists of Kernels 5 and 6. The largest performance jump appears between Kernel 4 and Kernel 5. It is a result of caching the field variable to shared memory. Kernel 5 appears to be the best performing kernel. As a result of succesively reducing the number of shared memory fetches, we improved the performance by a factor of two, from approximately 300 GFLOPS/s to around 600 GFLOPS/s.

**Table 6.** Details of performance analysis for BK5.0 and $N = 11$ on a mesh with 4096 elements.[a]

| | Registers | Shared (kB) | Shared load (GB/s) | Shared store (GB/s) | Stall reason | GFLOPS/s | % of roofline |
|---|---|---|---|---|---|---|---|
| Kernel 1 | 88 | 14.625 | 280.40 | 10.88 | ED (90.52%) | 193.49 | 22.0 |
| Kernel 2 | 156 | 14.625 | 686.65 | 26.76 | DR (79.49%) | 472.82 | 53.6 |
| Kernel 3 | 88 | 14.625 | 280.70 | 10.9 | DR (90.85%) | 193.34 | 21.9 |
| Kernel 4 | 154 | 14.625 | 726.41 | 28.21 | DR (73.64%) | 498.41 | 56.5 |
| Kernel 5 | 100 | 3.3750 | 2688.7 | 72.93 | DR (49.02%) | 673.15 | 76.4 |
| Kernel 6 | 80 | 16.875 | 4760.7 | 137.44 | DR (54.33%) | 856.64 | 97.2 |
| Kernel 7 | 62 | 16.875 | 4363.4 | 125.97 | DR (53.72%) | 783.26 | 88.9 |
| Kernel 8 | 62 | 16.875 | 4348.1 | 125.53 | DR (53.45%) | 783.21 | 88.9 |
| Kernel 9 | 88 | 16.875 | 4189.5 | 134.84 | DR (56.02%) | 842.51 | 95.6 |
| Kernel 10 | 94 | 4.5000 | 4432.5 | 144.16 | DR (49.59%) | 901.35 | 102.3 |

ED: execution dependency; DR: waiting for data request.

[a]The *stall reason* column shows the most common reason for stalls. The best performing kernel, kernel 10, uses very little shared memory but uses it effectively (we perform two FLOPS per every 8 bytes loaded from the shared memory). High number of registers and low occupancy does not prevent the kernel from achieving high FLOP count.

**Algorithm 7.** BK5.0: Collocation differentiation for 3D Hexahedral Mesh (3D Thread Structure).

**Data:** (1) Vector $\mathbf{q}$, size $N_{el} \times N_p$, (2) differentiation matrix $\tilde{\mathbf{D}}^{1\mathrm{D}}$, size $N_q \times N_q$, (3) geometric factors $\mathbf{G}$, size $N_{el} \times N_p^{GL} \times 7$, (4) parameter $\lambda$ ;
**Output:** Vector $\mathbf{Sq}$, size $N_{el} \times N_p$ ;
**for** For $e \in \{1, 2, \ldots, N_{el}\}$ **do**
  **for** $i, j, k \in \{1, 2, \ldots N_q\}$ **do**
    If `k=0`, load $\mathbf{D}^{1\mathrm{D}}$ to shared memory variable `s_D`;
    Declare register variables `r_qr`, `r_qs`, and `r_qt`;
  **end for** ▷ Synchronize threads.
  **for** $i, j, k \in \{1, 2, \ldots N_q\}$ **do**
    Load `GwJ`;
    Declare variables `qr`, `qs`, `qt` and set them to `0`.
    `qr` = $\sum_{n=1}^{N_q}$ `s_D`$_{in}$ $*$ $\mathbf{q}^e_{kjn}$;
    `qs` = $\sum_{n=1}^{N_q}$ `s_D`$_{jn}$ $*$ $\mathbf{q}^e_{kni}$;
    `qt` = $\sum_{n=1}^{N_q}$ `s_D`$_{kn}$ $*$ $\mathbf{q}^e_{nji}$;
    Set `r_qr = qr`, `r_qs = qs` and `r_qt = qt`;
    $\mathbf{Sq}^e_{kji}$ =`GwJ`*$\lambda$ $*$ $\mathbf{q}^e_{kji}$;
  **end for** ▷ Synchronize threads.
  **for** $i, j, k \in \{1, 2, \ldots N_q\}$ **do**
    Load `G00`, `G01`, `G02`;
    `Sqtemp`$_{kji}$= `G00*r_qr + G01*r_qs + G02*r_qt`;
  **end for**
  **for** $i, j, k \in \{1, 2, \ldots N_q\}$ **do**
    $\mathbf{Sq}^e_{kji}$ = $\mathbf{Sq}^e_{kji}$ + $\sum_{n=1}^{N_q}$ `s_D`$_{ni}$ $*$ `Sqtemp`$_{kjn}$;
  **end for** ▷ Synchronize threads.
  **for** $i, j, k \in \{1, 2, \ldots N_q\}$ **do**
    Load `G10`, `G11`, `G12`;
    `Sqtemp`$_{kji}$ = `G10*r_qr + G11*r_qs + G12*r_qt`;
  **end for** ▷ Synchronize threads.
  **for** $i, j, k \in \{1, 2, \ldots N_q\}$ **do**
    $\mathbf{Sq}^e_{kji}$ = $\mathbf{Sq}^e_{kji}$ + $\sum_{n=1}^{N_q}$ `s_D`$_{nj}$ $*$ `Sqtemp`$_{kni}$;
  **end for** ▷ Synchronize threads.
  **for** $i, j, k \in \{1, 2, \ldots N_q\}$ **do**
    Load `G10`, `G11`, `G12`;
    `Sqtemp`$_{kji}$ = `G20*r_qr + G21*r_qs + G22*r_qt`;
  **end for** ▷ Synchronize threads.
  **for** $i, j, k \in \{1, 2, \ldots N_q\}$ **do**
    $\mathbf{Sq}^e_{kji}$ = $\mathbf{Sq}^e_{kji}$ + $\sum_{n=1}^{N_q}$ `s_D`$_{nk}$ $*$ `Sqtemp`$_{nji}$;
  **end for**
**end for**

**Table 7.** Optimization strategies applied to a baseline kernel with 3-D thread structure for BK5.0 problem.

| Kernel # | Type of optimization |
|---|---|
| Kernel 0 | Baseline, no optimization. |
| Kernel 1 | All input variables declared as constant; loops with *n* iterator unrolled. |
| Kernel 2 | Geometric factors fetched only once and stored in registers. |
| Kernel 3 | Register variables used for partial results. |
| Kernel 4 | Variable **q** loaded to the shared memory. |
| Kernel 5 | Allocated three extra shared memory arrays to store partial results. |

## Optimizing Benchmark Problem 3.0

### *BK3.0 kernel design*

BK3.0 can be considered a fusion of BK1.0 and BK5.0. This benchmark shares its interpolation/projection steps with BK1.0 and the stiffness matrix action with BK5.0. Hence, BK3.0 inherits all the optimization challenges we have discussed thus far for BK1.0 and BK5.0. In particular, the kernel for this problem needs to synchronize threads multiple times and needs to load a large amount of data from global memory. Furthermore, compared to either BK1.0 or BK5.0 this benchmark requires even more shared memory usage in order to store partial results. Since differentiation is performed using a denser set of nodes, we also transfer more data per thread block compared with BK5.0.

The action of the screened Poisson operator in this benchmark can be written as three distinct parts: interpolation to GL nodes, stiffness and mass matrix actions, and projection back to GLL nodes. Therefore it is possible to split the implementation into three kernels. This approach reduces memory requirements per kernel and makes the code more readable. We investigate this approach with 3-D thread structure.

### *BK3.0 kernel optimization: 2-D thread structure*

We begin as in BK5.0 by first investigating a kernel implementation using a 2-D thread structure. The

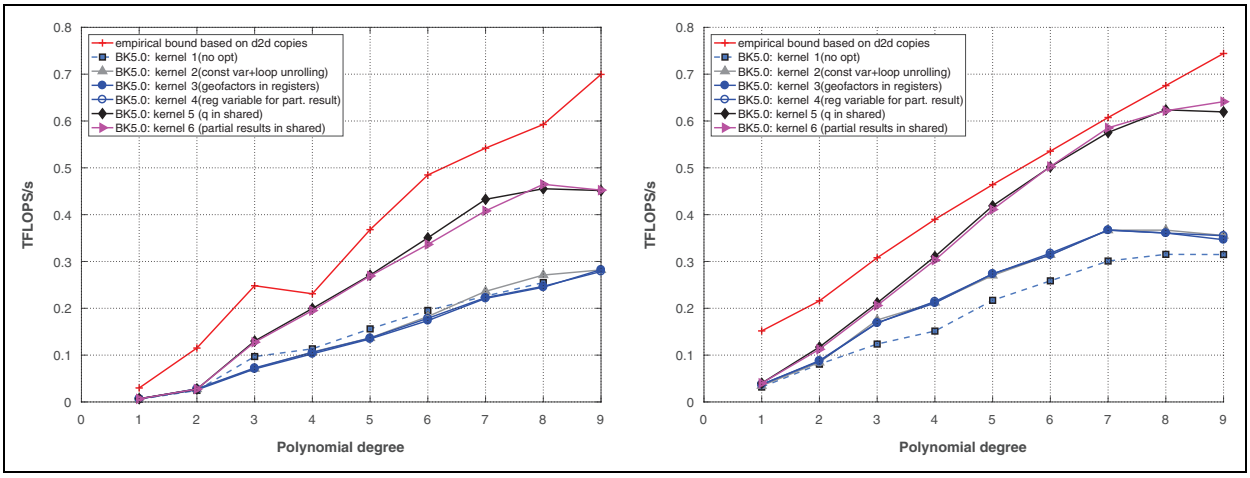


**Figure 8.** BK5.0: Performance of 3-D kernels in various stages of optimization. The red line marked with crosses is the roofline computed based on device-to-device copies measured on an NVIDIA P100 PCI-E 12-GB GPU. Left: TFLOPS/s for cubical mesh with 512 elements. Right: TFLOPS/s for cubical mesh with 4096 elements. GPU: graphics processing unit.

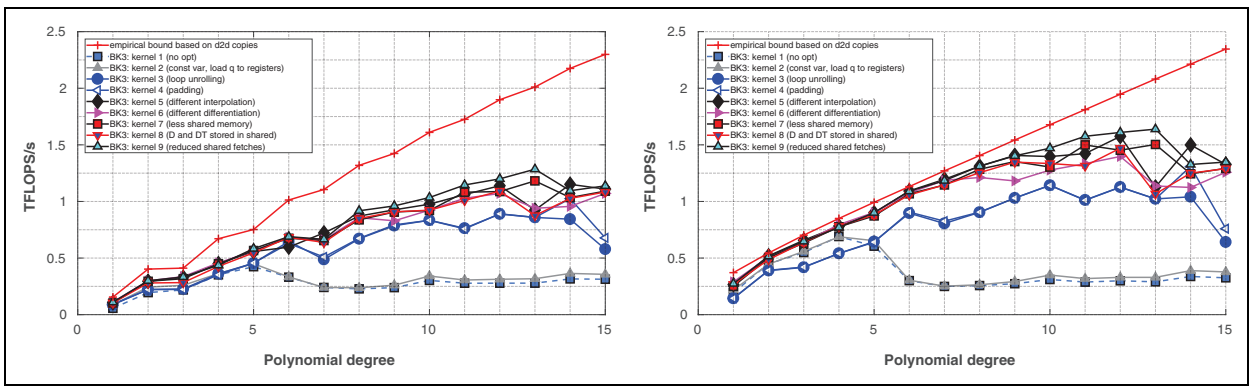


**Figure 9.** BK3.0: Performance of 2-D thread array kernels in various stages of optimization. The red line marked with crosses is the roofline computed based on device-to-device copies on a single NVIDIA P100 PCI-E 12-GB GPU. Left: TFLOPS/s for cubical mesh with 512 elements. Right: TFLOPS/s for cubical mesh with 4096 elements.

performance results for nine separate kernel are presented in Figure 9. As in the previous benchmarks, we present a sequence of kernels and detail what optimizations we performed in each kernel.

*Kernel 1*. This kernel serves as a reference implementation. In this kernel, there are four shared memory arrays: one for $\mathbf{I}^{1D}$, one for $\mathbf{D}^{1D}$, and two arrays with $(N_q^{GL})^2$ elements for storing partial sums. Each thread also allocates an additional register array to store additional partial sums that do not require sharing between threads in a thread block. The field variable $\mathbf{q}^e$ is read directly from global memory when needed. This kernel reaches approximately 280 GFLOPS/s.

*Kernel 2*. In this kernel, all the input variables, except the pointer used for storing the final results, are declared as `const`. Each thread allocates a register array of size $N_q$ and caches a column of entries of $\mathbf{q}^e$ to the register array. The performance for $N \geq 12$ for the larger mesh improves to over 1.25 TFLOP/s

*Kernel 3*. In this kernel, all internal loops are unrolled. The performance improves to 600 GFLOPS/s for the smaller mesh and to 1.4 TFLOP/s for the larger mesh.

*Kernel 4*. In this kernel, we add padding to all the shared memory arrays to avoid bank conflicts. In particular, in case $N_q^{GL} = 8$ or $N_q^{GL} = 16$, the arrays with size $N_q^{GL} \times N_q^{GL}$ change their size to $N_q^{GL} \times (N_q^{GL} + 1)$. Padding helps only for the smaller mesh.

*Kernel 5*. This kernel adapts the most efficient interpolation strategy devised for BK1.0 (see BK1.0: Kernel 7 and Figure 4). For the larger mesh, the performance reaches 1.5 TFLOP/s.

*Kernel 6*. In this kernel, the field variable $\mathbf{q}^e$ is loaded to a shared memory array. This kernel implements more efficient

**Table 8.** Optimization strategies applied to a baseline kernel with 2-D thread structure for BK3.0 problem.

| Kernel # | Type of optimization |
|---|---|
| Kernel 1 | Baseline, no optimization. |
| Kernel 2 | Kernel 1 + input variables are declared as constant. We load **q** to register array to eliminate global accesses. |
| Kernel 3 | Kernel 2 + loops involving *k* and *n* are unrolled. |
| Kernel 4 | Kernel 3 + all shared memory arrays are padded for $N_q^{GL} = 8.16$ and $N_q = 8.16$. |
| Kernel 5 | The interpolation and projection are changed to what we developed in kernel 10 for BK1.0. As a result, we reduce the number of synchronizations. |
| Kernel 6 | The differentiation strategy changes. |
| Kernel 7 | We eliminate one of the shared memory arrays used as placeholder for partial result on the cost of extra synchronization. |
| Kernel 8 | Both D and its transpose DT are stored in shared memory. |
| Kernel 9 | We use the same approach as in BK1.0, kernel 10, to reduce shared memory transactions. |

differentiation (as in BK5.0: Kernel 9). The improvement is very modest since in order to use this type of differentiation we need to combine it with less efficient interpolation.

*Kernel 7.* A version of Kernel 6 with one less shared memory array. We use only one shared memory array to store the partial result at a cost of additional synchronizations. Kernel 7 performs better than Kernel 6; however, in most cases, it is not more efficient than Kernel 5.

*Kernel 8.* This kernel is a version of Kernel 7 with both $\mathbf{D}^{1D}$ and $(\mathbf{D}^{1D})^T$ fetched to shared memory. Now all the threads can access shared memory column-wise. Performance is very similar to Kernel 6.

*Kernel 9.* The kernel uses the same strategy as in the last kernel implemented for BK1.0, that is, it fetches each repeating entry of the interpolation matrix $\mathbf{I}^{1D}$ only once. The performance of this kernel is the best that we present here as it reaches 1.6 TFLOPS/s. The optimization strategies are summarized in Table 8.

## BK3.0 results: 2-D thread structure

For this benchmark problem, our best performing kernel for high-order FEM approximations, Kernel 9, performs approximately four times as fast as the reference kernel, Kernel 1. Although we did not achieve the peak performance as predicted by our empirical roofline model using global memory bandwidth, for the 4096 element mesh and $N \leq 12$ our kernels are very close to that peak.

Output from the NVIDIA profiler suggest that the best kernel, Kernel 9, suffers from execution dependency stalls. As emphasized previously, the kernel has three distinct parts that have to be executed in an order, and hence, this finding is not surprising. The shared memory bandwidth for Kernel 9 is very high; in fact, much higher than for Kernel 10 for BK1.0, which uses the same interpolation strategy.

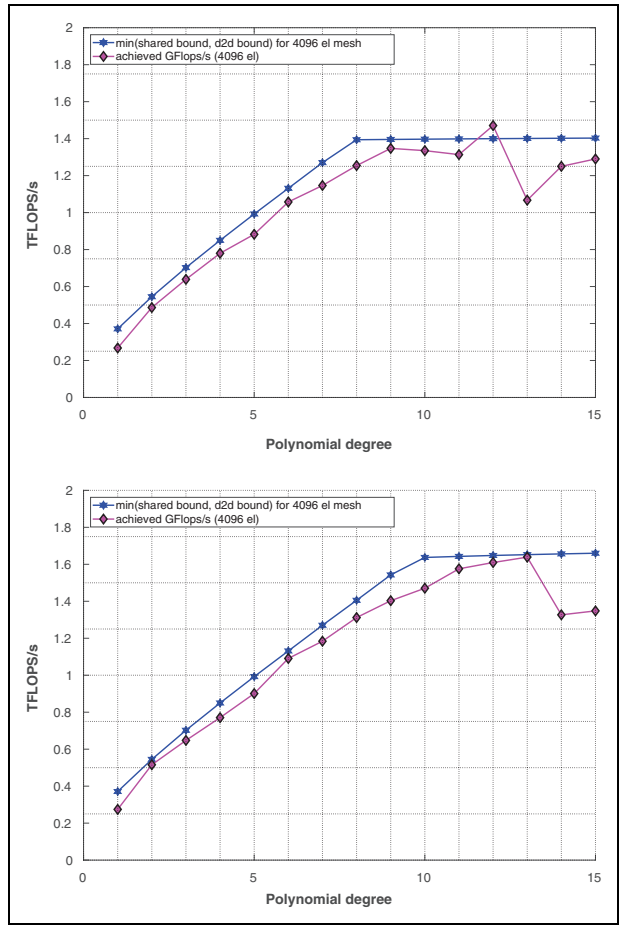


**Figure 10.** BK3.0: Shared memory rooflines for kernels 8 and 9. Top: Achieved floating point performance and shared memory roofline for kernel 8. Bottom: Achieved floating point performance and shared memory roofline for kernel 9.

As in BK1.0, we can obtain a better understanding of the performance of our kernels by considering the roofline model based on shared memory bandwidth. Figure 10 shows updated rooflines for Kernels 8 and 9 in which we see that this shared memory bandwidth bound yields a better estimate for the achieved performance of these kernels with $N \geq 10$.

Table 9 presents performance details for all the kernels with $N = 11$. We use $N = 11$ because our code achieved close to peak performance for this degree. The table shows differences in resource utilization for all the kernels.

## BK3.0 3-D thread structure

We perform analogous tuning process using a 3-D thread structure for $N < 9$. Figure 11 shows the performance of these kernels presented below.

**Table 9.** Details of performance analysis for BK3.0 and $N = 11$ on a mesh with 4096 elements.

| | Registers | Shared (kB) | Shared load (GB/s) | Shared store (GB/s) | Stall reason | GFLOPS/s | % of roofline |
|---|---|---|---|---|---|---|---|
| Kernel 1 | 64 | 6.5000 | 1761.2 | 52.27 | DR (43.25%) | 277.53 | 16.8 |
| Kernel 2 | 64 | 6.5000 | 1930.7 | 57.30 | ED (49.27%) | 305.00 | 18.4 |
| Kernel 3 | 80 | 6.5000 | 6368.0 | 189.0 | ED (61.65%) | 1003.73 | 60.6 |
| Kernel 4 | 80 | 6.5000 | 6322.7 | 179.95 | DR (33.36%) | 1010.57 | 61.0 |
| Kernel 5 | 88 | 22.344 | 6462.8 | 388.78 | ED (60.29%) | 1418.73 | 85.6 |
| Kernel 6 | 96 | 22.344 | 5610.3 | 366.88 | ED (49.27%) | 1330.23 | 80.3 |
| Kernel 7 | 86 | 21.023 | 6806.5 | 443.73 | ED (53.77%) | 1497.82 | 90.4 |
| Kernel 8 | 88 | 22.344 | 5959.7 | 361.38 | ED (50.82%) | 1307.93 | 78.9 |
| Kernel 9 | 112 | 21.023 | 6468.2 | 430.42 | ED (45.22%) | 1572.44 | 94.9 |

ED: execution dependency; DR: waiting for data request.
[a]The *stall reason* column shows the most common reason for stalls. The best performing kernel, kernel 9, optimizes shared memory read transactions. The achieved shared memory read bandwidth drops, but the number of transactions is halved.

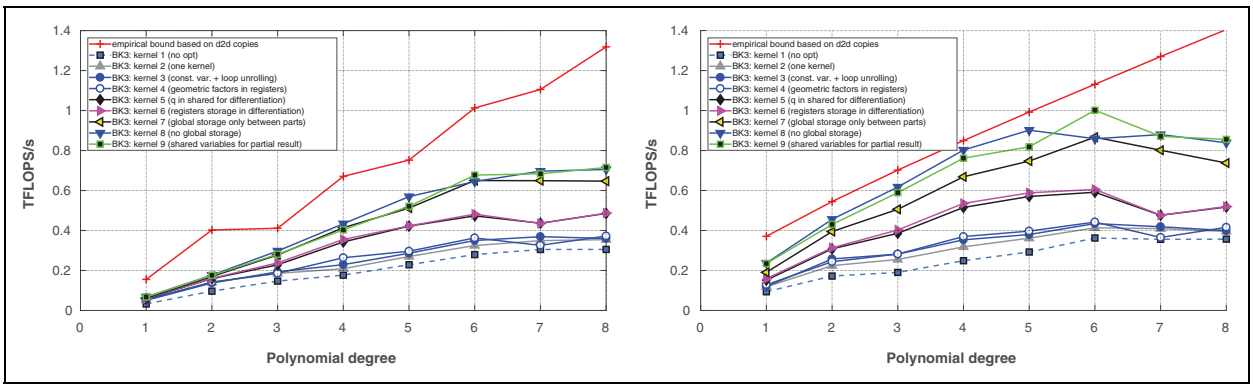


**Figure 11.** BK3.0: Performance of 3-D kernels in various stages of optimization. The red line marked with crosses is the roofline computed based on device-to-device copies on a single NVIDIA P100 PCI-E 12-GB GPU. Left: TFLOPS/s for cubical mesh with 512 elements. Right: TFLOPS/s for cubical mesh with 4096 elements. Note: almost all kernels fail for $N = 3$ on the smaller mesh. GPU: graphics processing unit.

*Kernel 1.* This kernel starts from a direct implementation of the pseudo-code given in Algorithm 5, which is executed as three separate CUDA kernels. The field variable $\mathbf{q}^e$ is read directly from global memory and partial results are stored in global memory as well. The code reaches approximately 280 GFLOPS/s in the best case.

*Kernel 2.* This kernel combines the three separate kernels as one CUDA kernel. There is almost no difference between Kernel 1 and Kernel 2. This suggests that the cost of additional kernel launches is small compared with the cost of data transfer.

*Kernel 3.* In this kernel, all internal loops are unrolled. Unrolling loops would appear to be more important for 2-D thread structure as for 3-D thread structure it has only a marginal influence on performance.

*Kernel 4.* This kernel differs from Kernel 2 by loading the geometric factors once and avoiding redundant reads. The gain in performance is small because of large amount of global memory transactions that have not yet been eliminated.

*Kernel 5.* While we access the field variable $\mathbf{q}^e$ directly from global memory in the interpolation and projection parts, $\mathbf{q}^e$ is fetched to a shared array in the differentiation step.

*Kernel 6.* This kernel is a version of Kernel 5 with intermediate results stored in registers in the differentiation step. Kernels 5 and 6 are approximately 1.5 times faster than Kernel 4.

*Kernel 7.* In this kernel, we store the partial results in shared memory and registers everywhere except between interpolation, first and second differentiation, and projection.

*Kernel 8.* In this kernel, there is no intermediate global storage. All intermediate results are accumulated either in shared memory or in registers.

*Kernel 9.* In this kernel, we declare three additional shared memory arrays (size $N_p$ each) for storing intermediate results. As a result, the performance reaches 900 GFLOPS/s. The optimization strategies are summarized in Table 10.

**Table 10.** Optimization strategies applied to a baseline kernel with 3-D thread structure for BK3.0 problem.

| Kernel # | Type of optimization |
|---|---|
| Kernel 1 | Baseline, no optimization. The kernel is executed as three separate CUDA kernels. |
| Kernel 2 | Kernel 1 combined into one CUDA kernel. |
| Kernel 3 | Kernel 2 + loop unrolling. |
| Kernel 4 | Kernel 3 + geometric factors are loaded only once and stored in registers. |
| Kernel 5 | Kernel 4 + **q** is fetched to the shared memory in the differentiation step (but not in interpolation and projection). |
| Kernel 6 | Kernel 5 + intermediate results stored in registers in the differentiation part. |
| Kernel 7 | Kernel 6 + all partial results are stored in shared memory or registers – except the results between interpolation and differentiation and differentiation and projection. |
| Kernel 8 | Kernel 7 + all partial results are stored either in shared memory or in registers. |
| Kernel 9 | Kernel 8 + three additional shared memory arrays for partial results. |

### *BK3.0 Results: 3-D thread structure*

Unlike in the case of BK5.0, in BK3.0 we observe a very gradual improvement in performance. However, as for BK5.0 the largest improvements appear when removing additional global memory transactions. It is interesting to note that the difference between Kernel 7 and Kernel 8 is rather small, despite the fact that Kernel 7 reads and writes three vectors of size $N_{el} \cdot N_p$ to the global memory and Kernel 8 reads and writes only once. The lack of significant improvement may be a result of the additonal synchronizations enforced in Kernel 8. In Kernel 7, the global write serves as synchronization.

## Conclusions and future work

In this article, we described the GPU optimization of three FEM-specific benchmark problems which have significant relevance in real-world large-scale code bases. We detailed how reducing factors such as global memory transactions and reducing pressure on shared memory usage can lead to efficient GPU kernels for these FEM operators. For BK5.0, the most-tuned kernel is aligned with the empirical roofline bound and its performance is limited only by global memory bandwidth. We obtained similar results for the two remaining problems but their performance does not scale perfectly for $N \geq 13$. For each of the problems, we used a sequence of optimizations. One might argue that multiple authors (see, e.g. Abdelfattah et al., 2016; Garvey and Abdelrahman, 2015; Nelson et al., 2015) consider automatic performance tuning and our efforts should rather be replaced by a computational tuning process. However, in each of the benchmarks we consider large performance improvements are gained by changing the algorithm itself, which is not an automatic process. While the performance bottlenecks such too many barriers in BK1.0 and high data fetch to FLOP ratio in BK3.0 and BK5.0 are often not hard to identify, they can be troublesome to remove. We use a mixture of standard optimization strategies such as splitting the data between registers and shared memory and more advanced strategies, which resulted in redesigning the algorithm to make it more well-suited for fine-grain parallelism.

An obvious question remaining is how much more performance can be obtained in each of these benchmarks. In this article, we provide contributions toward the answer. Unlike Abdelfattah et al. (2016), we consider the limitation of shared memory bandwidth in our approach. By no mean our model is exhaustive and our future work involves introducing a more advanced performance model that accounts for register use and other factors but retains the simplicity of the original model. The source code for all kernels and benchmarks considered in this article is publicly available at: https://github.com/kswirydo/CEED-Ax.

The results presented in this article were obtained on an NVIDIA Tesla P100 GPU. One might ask whether we should expect similar optimizations to yield good performance on the newer generation of NVIDIA GPUs (Volta) or on older generation NVIDIA GPUs (Fermi, Kepler, and Maxwell), and if not, what performance can one expect. Pascal cards, such as the NVIDIA Tesla P100, have significantly more streaming multiprocessors than its predecessors with less computational cores per multiprocessor. This allows us to split the problem into more blocks with less threads per block. Since each streaming multiprocessor has its own L1 cache, we can rely more heavily on the L1 cache in our optimizations. Furthermore, Pascal cards have an improved ratio of double-precision to single-precision performance, yielding significant performance gains in the benchmarks presented. Thus, while running the code as is using double precision on older generation of NVIDIA GPUs, such as Kepler, which has fewer streaming multiprocessors and four times as many cores per multiprocessor, we expect the performance to be far from the roofline. On the other hand, the newer generation of Volta cards has architecture similar to Pascal cards but also come with unified L1 cache and shared memory space. This feature impacts the performance of kernels that rely heavily on both L1 and shared memory.

### Acknowledgements

This research was supported in part by the Exascale Computing Project, a collaborative effort of two US Department of Energy organizations (Office of Science and the National Nuclear Security Administration) responsible for the planning and preparation of a capable exascale ecosystem, including software, applications, hardware, advanced system engineering, and early testbed platforms, in support of the nation's exascale computing imperative.

In addition, the authors would like to kindly acknowledge Advance Research Computing at Virginia Tech for providing readily accessible computational resources. Finally, this research was supported in part by the John K. Costain Faculty Chair in Science at Virginia Tech.

While the manuscript was under revision, Kasia Świrydowicz was working in National Renewable Energy Laboratory in Golden, Colorado. This work was authored in part by the National Renewable Energy Laboratory, operated by Alliance for Sustainable Energy, LLC, for the US Department of Energy (DOE) under contract no. DE-AC36-08GO28308. The views expressed in the article do not necessarily represent the views of the DOE or the US Government. The publisher, by accepting the article for publication, acknowledges that the US Government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this work, or allow others to do so, for US Government purposes.

## Declaration of Conflicting Interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

## Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This research was supported in part by the Exascale Computing Project (17-SC-20-SC), a collaborative effort of two US Department of Energy organizations (Office of Science and the National Nuclear Security Administration) responsible for the planning and preparation of a capable exascale ecosystem, including software, applications, hardware, advanced system engineering, and early testbed platforms, in support of the nation's exascale computing imperative.

## Note

1. Unlike the copy-based empirical streaming roofline, the shared memory performance bound depends on the kernel, not on the problem.

## References

Abdelfattah A, Baboulin M, Dobrev V, et al. (2016) High-performance tensor contractions for GPUs. *Procedia Computer Science* 80: 108–118.

Cecka C, Lew AJ and Darve E (2011) Assembly of finite element methods on graphics processors. *International Journal of Numerical Methods in Engineering* 85(5): 640–669.

CEED (2017) CEED benchmark problems. Available at: http://ceed.exascaleproject.org/bps/ (accessed 6 November 2018).

Dehnavi MM, Fernández DM and Giannacopoulos D (2010) Finite-element sparse matrix vector multiplication on graphic processing units. *IEEE Transactions on Magnetics* 46(8): 2982–2985.

Deville MO, Fischer PF and Mund EH (2002) *High-Order Methods for Incompressible Fluid Flow*. Cambridge: Cambridge University Press.

Don WS and Solomonoff A (1995) Accuracy and speed in computing the Chebyshev collocation derivative. *SIAM* Journal of Scientific Computing 16(6): 1253–1268. DOI: 10.1137/0916073.

Dziekoński A, Rewieński M, Sypek P, et al. (2017) GPU-accelerated LOBPCG method with inexact null-space filtering for solving generalized eigenvalue problems in computational electromagnetics analysis with higher-order FEM. *Journal of Computational Physics* 22(4): 997–1014.

Fischer PF, Heisey K and Min M (2015) Scaling limits for PDE-based simulation. In: *Proceedings of 22nd AIAA computational fluid dynamics conference*, Dallas, United States, 22 June 2015.

Fischer PF, Lottes JW and Kerkemeier SG (2008) Nek5000 Web page. Available at: http://nek5000.mcs.anl.gov (accessed 6 November 2018).

Fu Z, Lewis TJ, Kirby RM, et al. (2014) Architecting the finite element method pipeline for the GPU. *Journal of Computational and Applied Mathematics* 257: 195–211.

Garvey JD and Abdelrahman TS (2015) Automatic performance tuning of stencil computations on GPUs. In: *Proceedings of 44th international conference on parallel processing*, Beijing, China, 1–4 September 2015, pp. 300–309. IEEE.

Göddeke D, Buijssen SH, Wobker H, et al. (2009) GPU acceleration of an unmodified parallel finite element Navier-Stokes solver. In: *Proceedings of international conference on High Performance Computing & simulation (HPCS'09)*, pp. 12–21. IEEE.

Göddeke D, Strzodka R, Mohd-Yusof J, et al. (2007) Exploring weak scalability for FEM calculations on a GPU-enhanced cluster. *Parallel Computing* 33(10-11): 685–699.

Göddeke D, Strzodka R and Turek S (2005) *Accelerating Double Precision FEM Simulations With GPUs*. Dortmund: Universität Dortmund, Fachbereich Mathematik.

Grigoras P, Burovskiy P, Luk W, et al. (2016) Optimising sparse matrix vector multiplication for large scale FEM problems on FPGA. In: *Proceedings of international conference on field programmable logic and applications (FPL 2016)*, pp. 1–9. IEEE.

Hong S and Kim H (2009) An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness. *SIGARCH Computer Architecture News* 37(3): 152–163. DOI: 10.1145/1555815.1555775.

Kim H, Vuduc R, Baghsorkhi S, et al. (2012) Performance analysis and tuning for general purpose graphics processing units (GPGPU). *Synthesis Lectures on Computer Architecture* 7(2): 1–96.

Konstantinidis E and Cotronis Y (2015) A practical performance model for compute and memory bound GPU kernels. In: *Proceedings of 23rd Euromicro international conference on parallel, distributed and network-based processing*, pp. 651–658. IEEE.

Lee VW, Kim C, Chhugani J, et al. (2010) Debunking the 100X GPU vs. CPU myth: an evaluation of throughput computing on

CPU and GPU. *SIGARCH Computer Architecture News* 38(3): 451–460. DOI: 10.1145/1816038.1816021.

Li A (2016) *GPU* performance modeling and optimization. PhD Thesis, Technische Universiteit Eindhoven.

Liu B, Wen C, Sarwate AD, et al. (2017) A unified optimization approach for sparse tensor operations on GPUs. In: *IEEE International conference on cluster computing (CLUSTER)*, Honolulu, HI, 5–8 September 2017, pp. 47–57. IEEE.

Lo YJ, Williams S, Van Straalen B, et al. (2014) Roofline model toolkit: a practical tool for architectural and program analysis. In: *Proceedings of international workshop on performance modeling, benchmarking and simulation of high performance computer systems*, pp. 129–148. Springer

Markall GR, Ham DA and Kelly PH (2010) Towards generating optimised finite element solvers for GPUs from high-level specifications. *Procedia Computer Science* 1(1): 1815–1823.

Markall GR, Slemmer A, Ham D, et al. (2013) Finite element assembly strategies on multi-core and many-core architectures. *International Journal for Numerical Methods in Fluids* 71(1): 80–97.

Medina DS, St-Cyr A and Warburton T (2014) OCCA: a unified approach to multi-threading languages. Available at: https://arxiv.org/abs/1403.0968 (2014, accesssed December 2018)

Nelson T, Rivera A, Balaprakash P, et al. (2015) Generating efficient tensor contractions for GPUs. In: *Proceedings of 44th international conference on parallel processing*, Beijing, China, pp. 969–978. IEEE.

Remacle JF, Gandham R and Warburton T (2016) GPU accelerated spectral finite elements on all-hex meshes. *Journal of Computational Physics* 324: 246–257.

Shi Y, Niranjan UN, Anandkumar A, et al. (2016) Tensor contractions with extended BLAS kernels on CPU and GPU. In: *Proceedings of 23rd international conference on High Performance Computing (HiPC)*, Piscataway, NJ, USA, 19–22 December 2016, pp. 193–202.

Stratton JA, Rodrigues C, Sung IJ, et al. (2012) *Parboil: A Revised Benchmark Suite for Scientic and Commercial Throughput Computing*: *Technical Report IMPACT-12-01*. University of Illinois at Urbana-Champaign.

Volkov V (2010) Better performance at lower occupancy. In: *Proceedings of the GPU technology conference, GTC*, San Jose, CA, 20–23 September 2010, volume 10, pp. 16. NVIDIA.

Volkov V and Demmel JW (2008) Benchmarking GPUs to tune dense linear algebra. In: *Proceedings of international conference for High Performance Computing, networking, storage and analysis (SC 2008)*, Austin, TX, USA, 15–21 November 2008, pp. 1–11. IEEE.

Wong H, Papadopoulou MM, Sadooghi-Alvandi M, et al. (2010) Demystifying GPU microarchitecture through microbenchmarking. In: *Proceedings of 2010 IEEE international symposium on performance analysis of systems software (ISPASS)*, pp. 235–246. DOI: 10.1109/ISPASS.2010.5452013.

Zhang Y and Owens JD (2011) A quantitative performance analysis model for GPU architectures. In: *Proceedings of 17th international symposium on high performance computer architecture*, pp. 382–393. DOI: 10.1109/HPCA.2011.5749745.

## Appendix

The results shown is this article can be reproduced by downloading the source code in the repository https://github.com/kswirydo/CEED-Ax. The source code has no dependency on external libraries other than OCCA, https://github.com/libocca/occa, which is also open-source.

### *What to expect*

The results vary depending on the hardware. For example, on an NVIDIA Tesla P100 and V100 we expect the results to be reasonably similar to the results presented in this article, but on Fermi and Kepler class cards the performance is far from the peak. Example performance plots for GPUs different than P100 can be found in the repository mentioned above. As the target architecture of CEED project, the code has been optimized for the NVIDIA Tesla P100 GPU and the user should be aware that performance is strongly dependent on the features of this and more modern GP-GPUs.

### Author biographies

*Kasia Świrydowicz* is a postdoctoral researcher in National Renewable Energy Laboratory. Her current research interests include code performance optimization for HPC systems and improving the scalability of Krylov subspace solvers.

*Noel Chalmers* is a software engineer at AMD Research. His research activities include developing and optimizing high performance computing applications on heterogeneous systems and GPU accelerators, and the development of scalable high-order finite element methods in elliptic problems as well as compressible and incompressible flows.

*Ali Karakus* is a postdoctoral researcher in the Department of Mathematics at Virginia Tech. His research interests include CFD, higher-order finite element methods and multiphase flows.

*Tim Warburton* currently holds the John K. Costain Faculty Chair in the College of Science, is a professor in the Department of Mathematics, and is also an associate faculty member in the Computational Mathematics and Data Analytics program at Virginia Tech. His research activities include developing and analyzing novel high-order finite element methods for solving PDEs. He has spent a decade developing GPU-accelerated discontinuous Galerkin methods for computational electromagnetics, acoustics, linear elasticity, elliptic problems, as well as compressible and incompressible flow solvers and mini-apps.